汽车电子与控制
Automotive Electronics and Control Systems (AECS)

李建秋、徐梁飞
清华大学车辆与运载学院
汽车安全与节能国家重点实验室
2020.9

第二章：汽车电控系统数字核心（单片机）
Digital Core of AECS (Microcontroller)

§目录 / Index

1 TC系列MCU结构与编程

2 TC系列MCU指令系统

3 TC系列MCU的CPU与外围电路

4 TC系列MCU的周边模块

1 TC系列MCU结构与编程

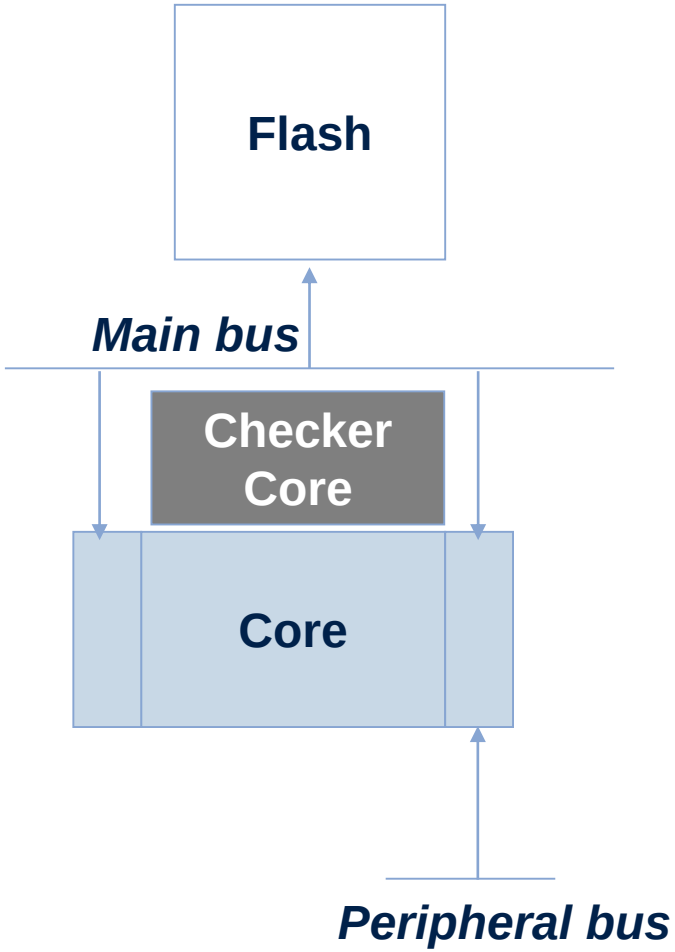
- (1) TC212L基本结构
- (2) 数据类型和(存储空间)寻址方式
- (3) CPU系统寄存器
- (4) 任务与现场
- (5) Interrupt System中断系统
- (6) Trap System 陷阱系统
- (7) CPU的通用和系统寄存器

学习方法:
掌握基本原理
结合实验
边用边学
查阅式阅读

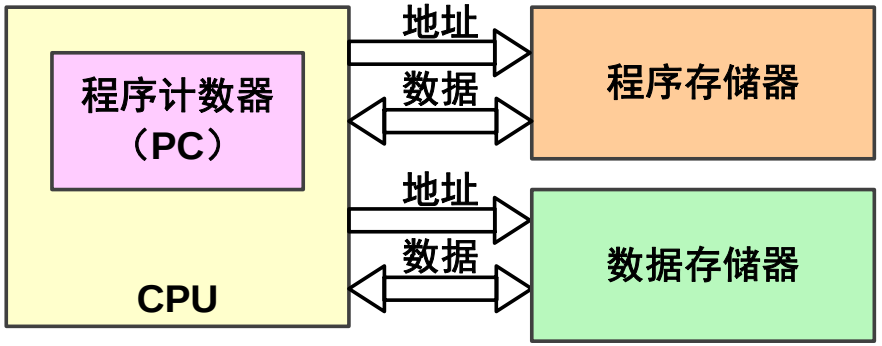
- 课外阅读文件:
  - 第2章参考资料
  - TC_Architecture_vol1_TC161_TCS_TC16P_TC16E
  - TC_Architecture_vol2_TC161_TCS_TC16P_TC16E
  - tc23x_tc22x_um_v1.1

(1) TC212L基本结构

AURIX Multi-Core Architecture



TC212L 哈佛体系结构



哈佛体系结构

(1) TC212L基本结构

锁步core

锁步核是实现高诊断覆盖度（检测错误发生的能力）的一种传统方法。
两个core同时运行相同的代码，并周期性比较两个core计算结果，若相同，则继续运行，否则，重新运行产生不同结果的代码再检查或重启。

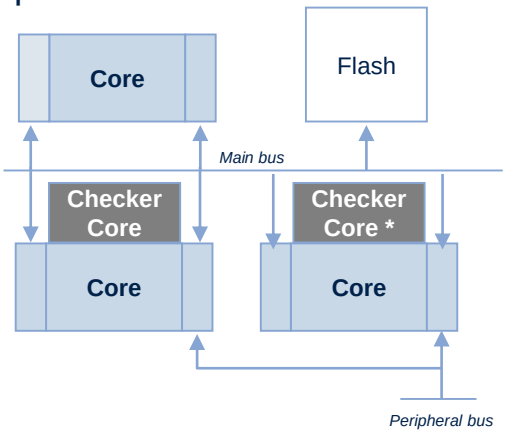
lock-step core在汽车芯片中常见。

(1) TC212L基本结构

Triple Core Lockstep

-

„**T**“ Marking

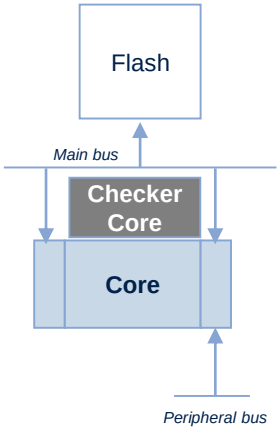


TC29xT and TC27xT

Lockstep

-

„**L**“ Marking

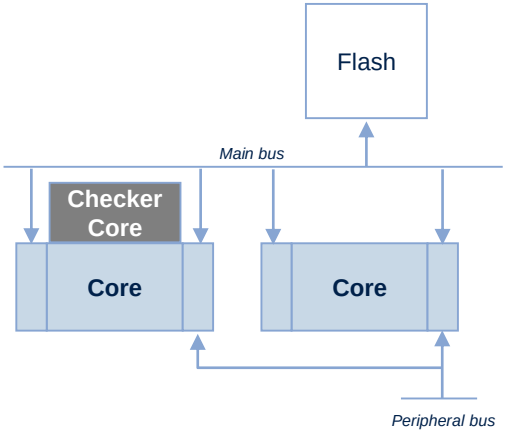


TC21xL,TC22xL and TC23xL

Dual Core Lockstep

-

„**D**“ Marking

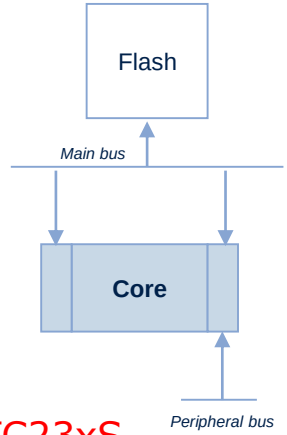


TC26xD

Single Core

-

„**S**“ Marking

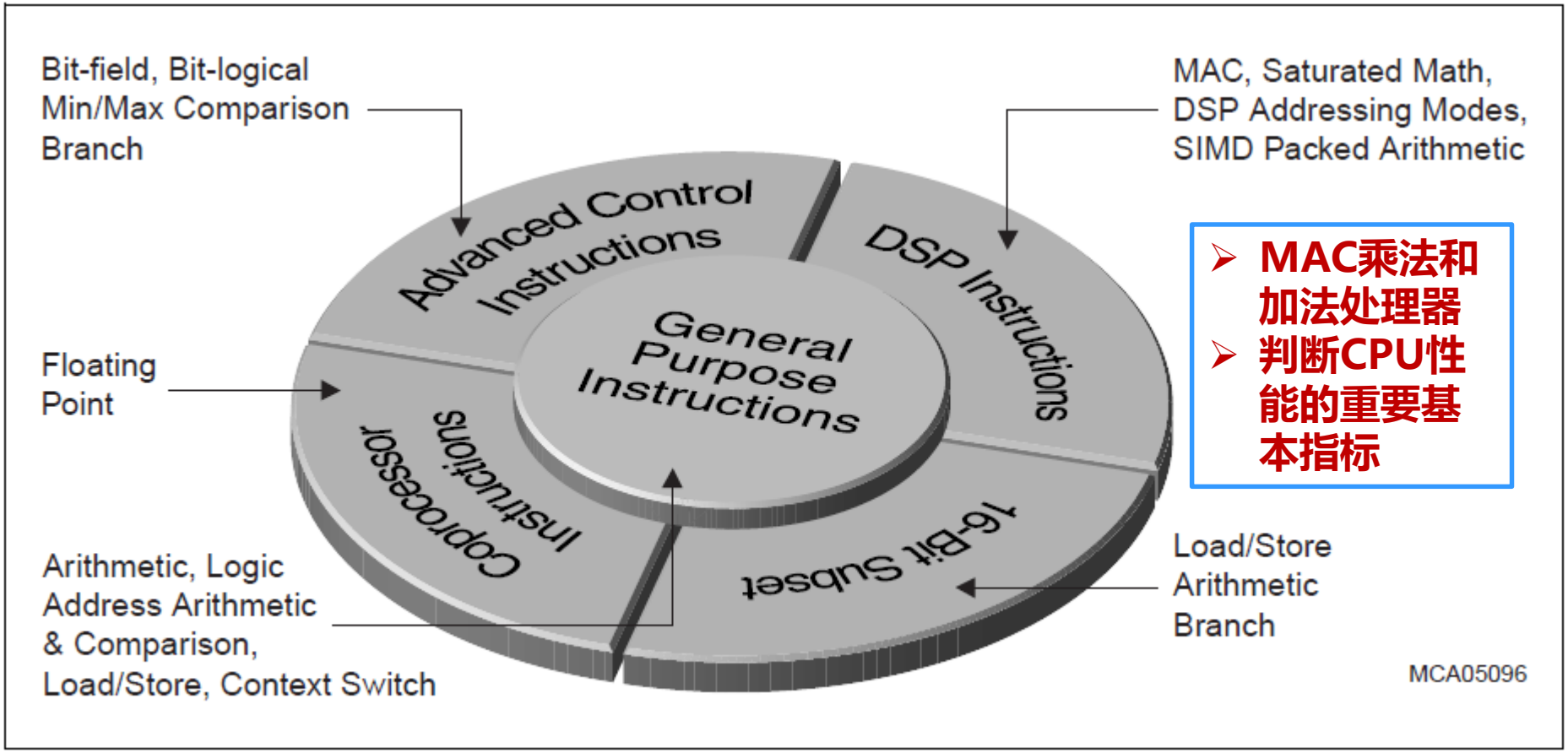


TC21xS,TC22xS and TC23xS

Copyright © Infineon Technologies AG 2015. All rights reserved.

(1) TC212L基本结构

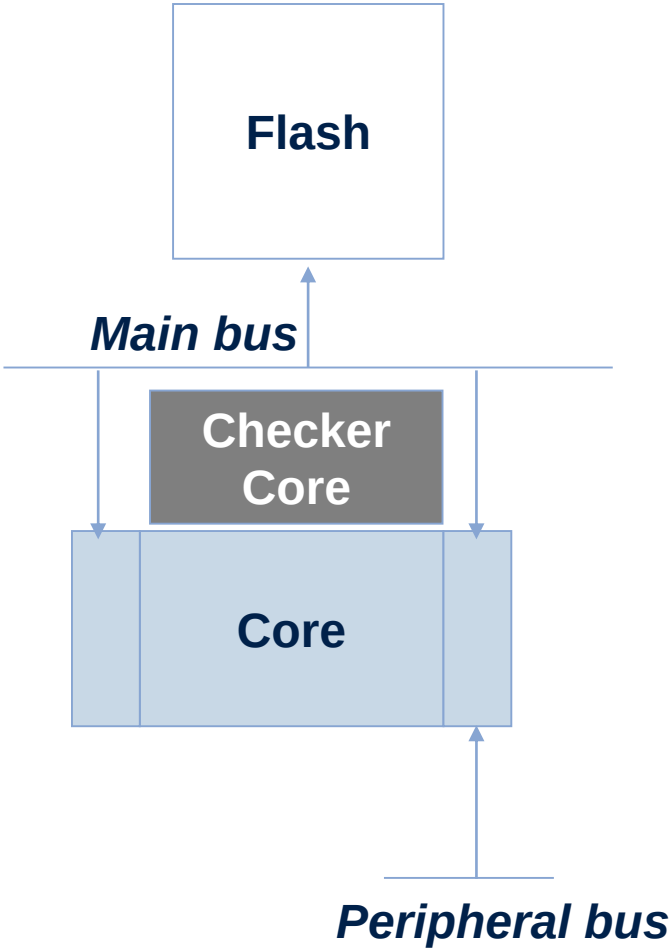
- TC2XX的CPU—TriCore® TC1.6P & TC1.6E



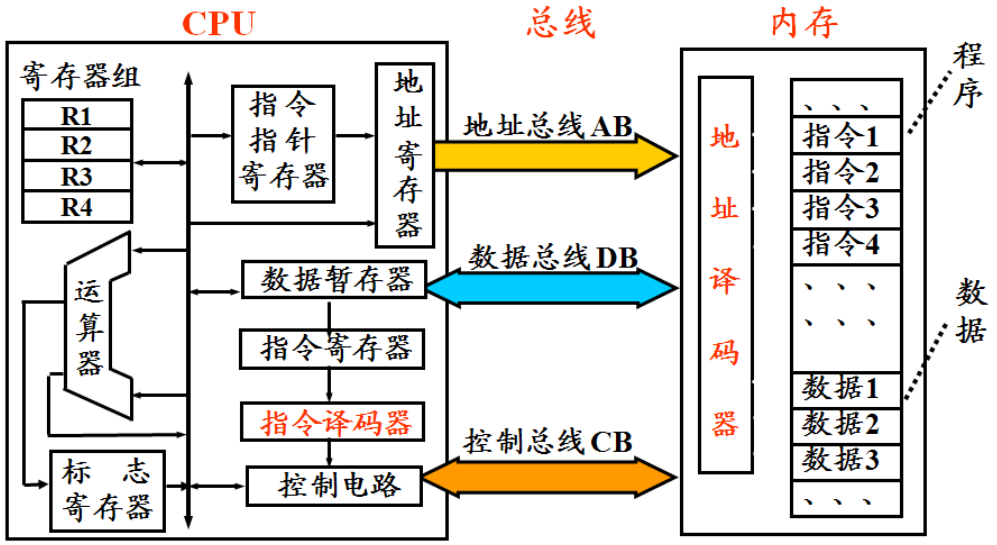
- MAC乘法和加法处理器
- 判断CPU性能的重要基本指标

Figure 1-1 TriCore Architecture Overview

(1) TC212L基本结构



TC212L 哈佛体系结构



普林斯顿结构

CPU：寄存器、指令、数据
存储空间：寄存器（数据，地址）、指令、数据

(1) TC212L基本结构

- (2) 数据类型和存储空间寻址方式
- (3) CPU系统寄存器
- (4) 任务与现场
- (5) Interrupt System中断系统
- (6) Trap System 陷阱系统
- (7) CPU的通用和系统寄存器

数据如何定义?
数据、指令、寄存器如何存储、如何寻址?

主要寄存器的意义
程序如何组织、如何运行?

(2) 数据类型和寻址方式

- 语法约定
  - ◆默认十进制，十六进制加下标H，二进制加下标B
  - ◆或者十六进制前面加$，二进制前面加%

128	128H	1101B
1101	$128	%1101

- ◆寄存器位访问

PSW.IS 表示访问PSW寄存器中的中断堆栈控制位

The Interrupt Stack Control bit of the PSW register.

(2) 数据类型和寻址方式

● 语法约定

Units are abbreviated as follows:

- ◆ – MHz = Mega Hertz.
- ◆ – kBaud, kBit = 1000 characters/bits per second.
- ◆ – MBaud, MBit = 1,000,000 characters per second.
- ◆ – KByte = 1024 bytes.
- ◆ – MByte = 1048576 bytes of memory.
- ◆ – GByte = 1,024 megabytes.

- **Data format quantities referenced are as follows:**
  - ◆ – Byte = 8-bit quantity.
  - ◆ – Half-word = 16-bit quantity.
  - ◆ – Word = 32-bit quantity.
  - ◆ – Double-word = 64-bit quantity.

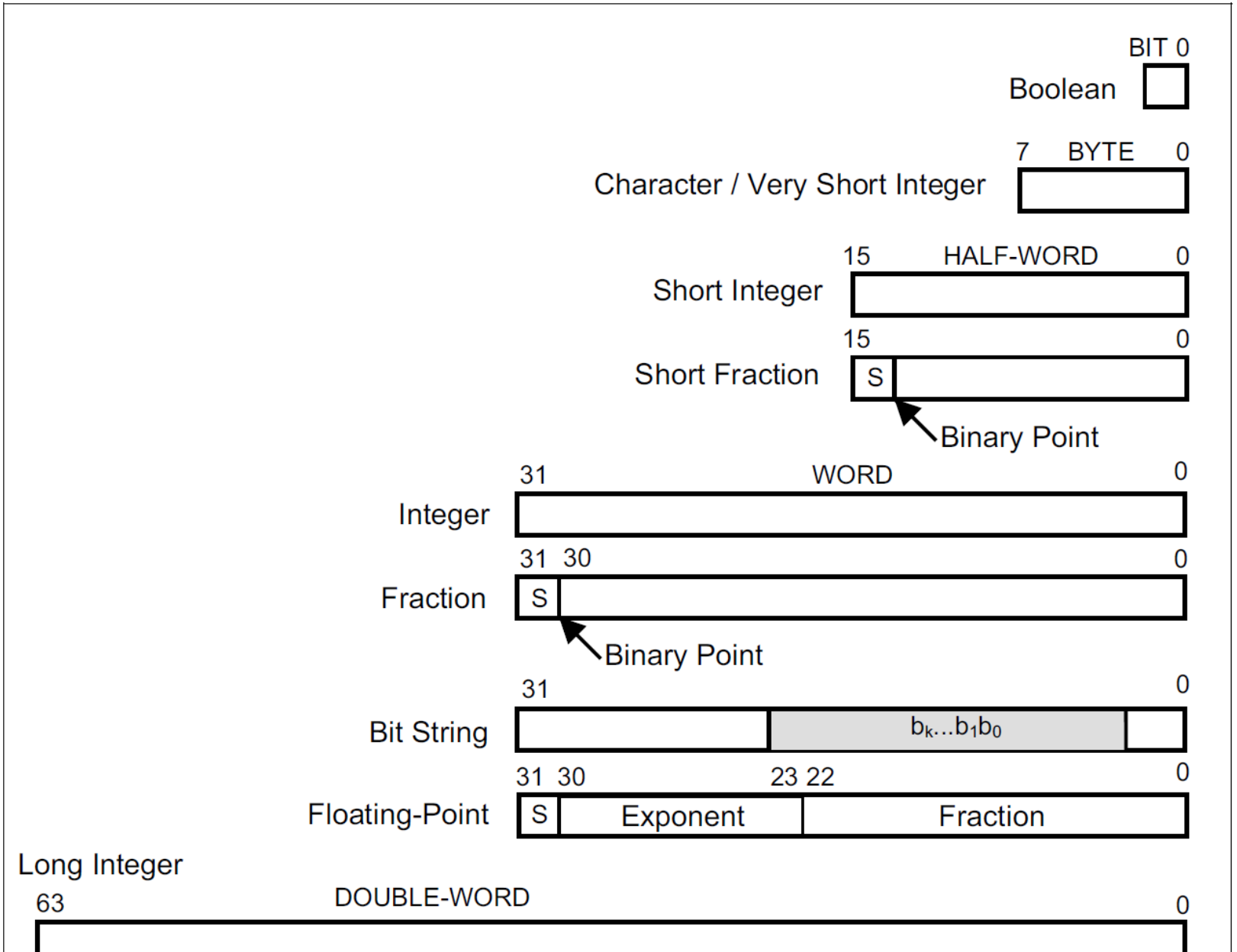
⇦ 注意：和普通计算机语言不同！

(2) 数据类型和寻址方式

- The instruction set supports operations on:
  - Boolean TRUE or FALSE
  - Bit String A bit string is a packed field of bits
  - Byte 8bit
  - Signed Fraction 16bit, 32bit, 64bit
  - Address 32 bit
  - Signed / Unsigned Integer 32bit
  - IEEE-754 Single-Precision Floating-Point（32位）

  Most instructions work on a specific data type, while others are useful for manipulating several data types.

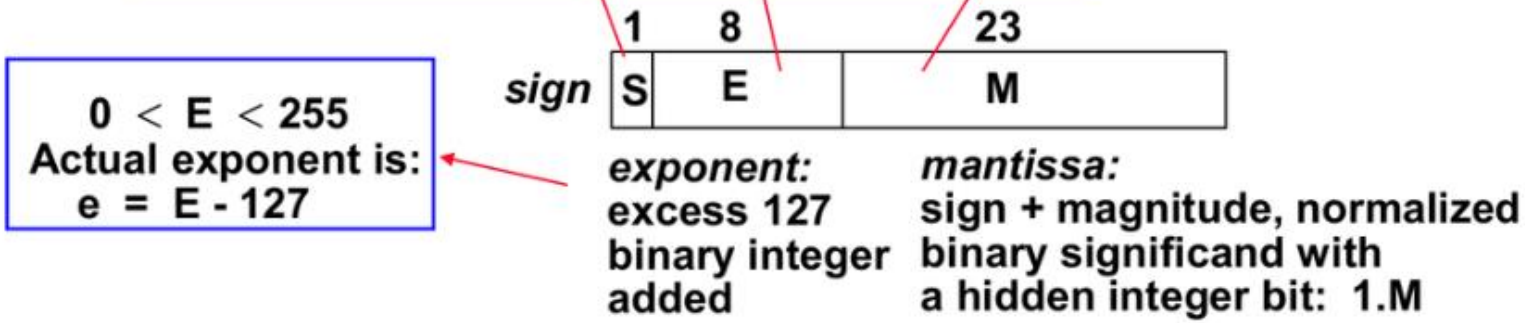
(2) 数据类型和寻址方式


BIT 0
Boolean
7 BYTE 0
Character / Very Short Integer
15 HALF-WORD 0
Short Integer
15 0
Short Fraction
S
Binary Point
31 WORD 0
Integer
31 30 0
Fraction
S
Binary Point
31 0
Bit String
$b_k \ldots b_1 b_0$
31 30 23 22 0
Floating-Point
S
Exponent
Fraction
Long Integer
63 DOUBLE-WORD 0

(2) 数据类型和寻址方式

Representation of Floating Point Numbers in Single Precision *IEEE 754 Standard*

$$\text{Value} = N = (-1)^S \times 2^{E-127} \times (1.M)$$



Example: 0 = 0 00000000 0 . . . 0 -1.5 = 1 01111111 10 . . . 0

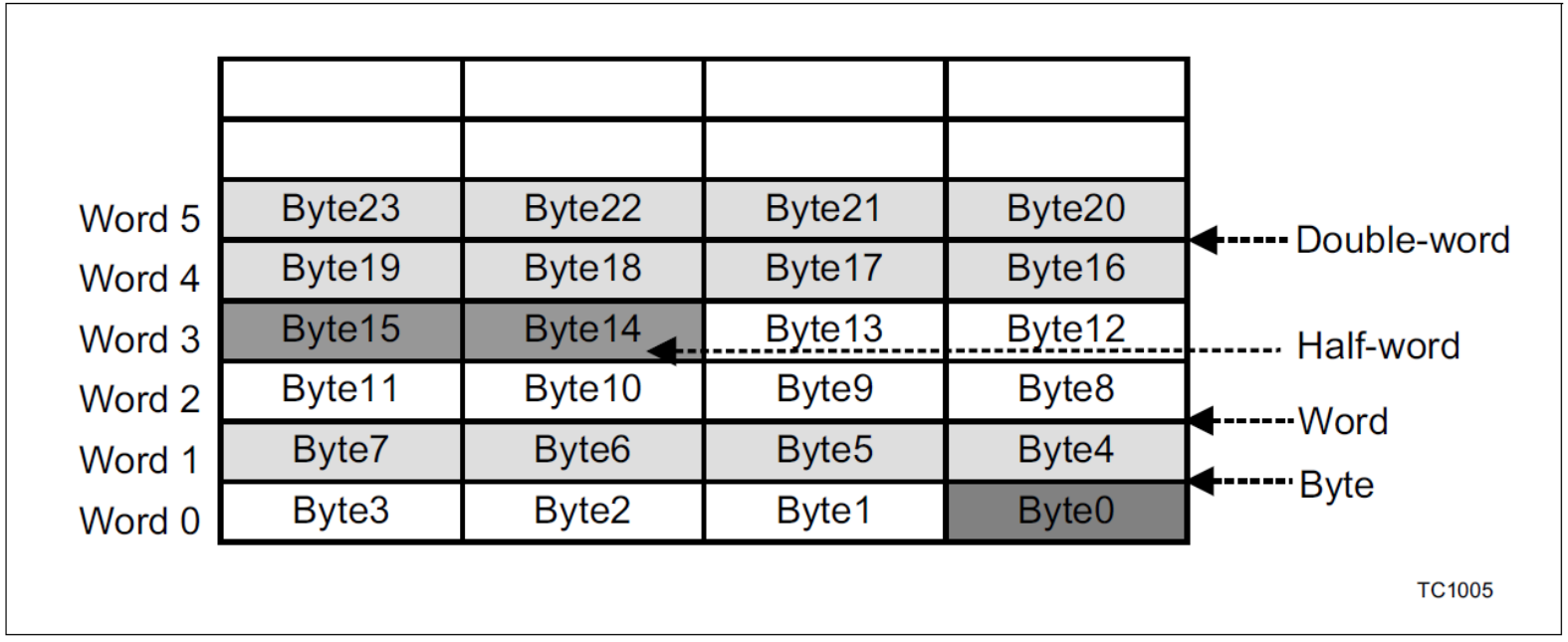
(2) 数据类型和寻址方式

- General Purpose Registers (GPRs):
  - ◆Word (32bit) 位基本运行单位
  - ◆少于32位运算时自动补齐32位
- 支持的数据格式
  - Bit
  - Byte: signed, unsigned
  - Half-word: signed, unsigned, fraction
  - Word: signed, unsigned, fraction, floating-point
  - 48-bit: signed, unsigned, fraction
  - Double-word: signed, unsigned, fraction

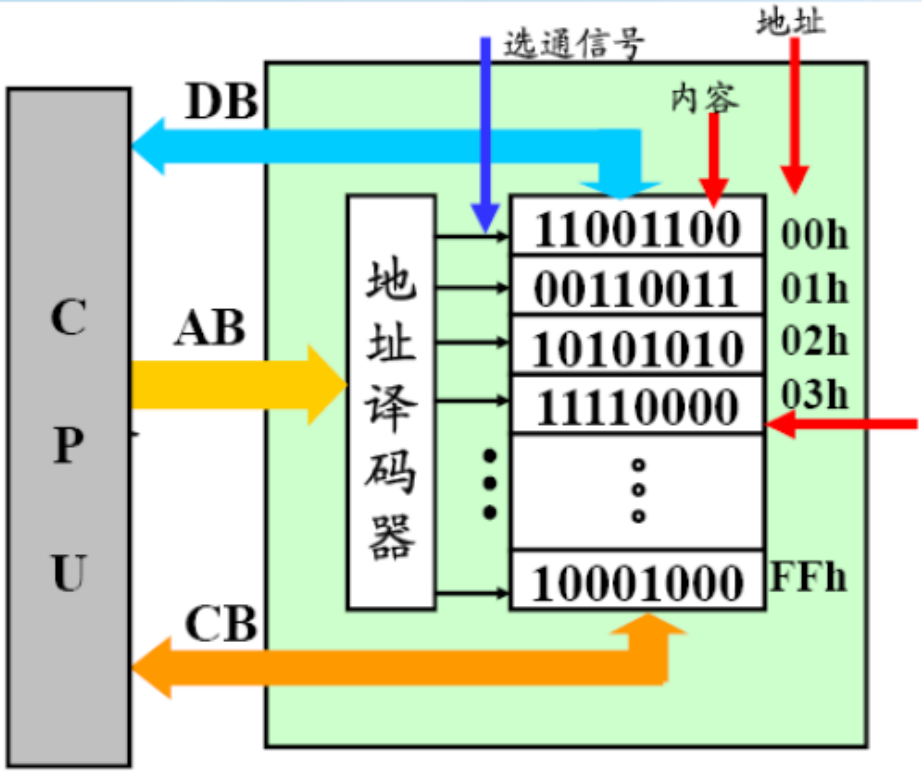
(2) 数据类型和寻址方式

- Byte order (字节存放顺序)
  - ◆little-endian byte order (the least-significant bytes are at lower addresses). （高位在高地址，低位在低地址，俗称高位在前，低位在后）
  - ◆Big-endian byte order

单片机中最小数据存储、访问单元为Byte



(2) 数据类型和寻址方式



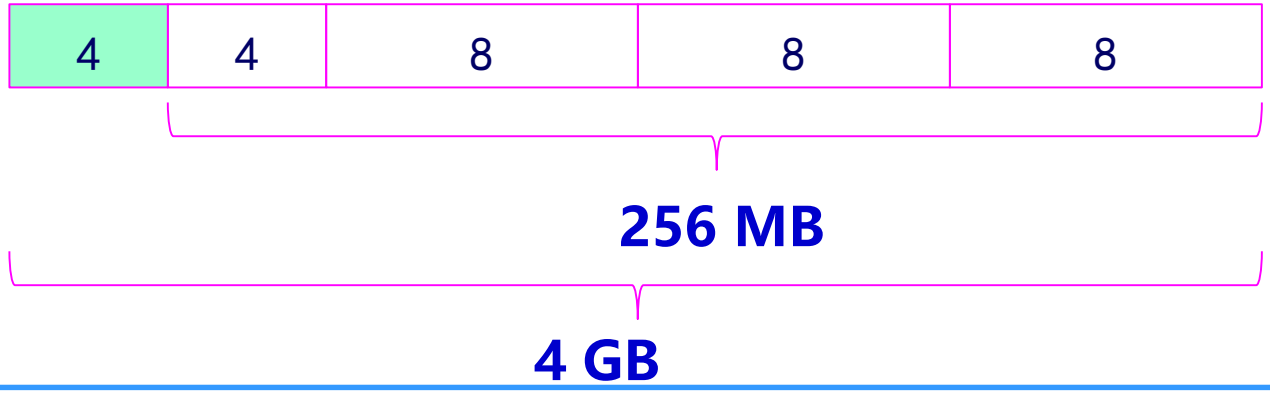
存储空间（内存）示意图：以普林斯顿结构为例

基本要素：寻址方式

TC2xx系列单片机地址范围：0000 0000H~FFFF FFFFH
每个地址对应存储空间（内存）中的一个Byte
并不是每款单片机都有4GByte 内存空间

(2) 数据类型和寻址方式

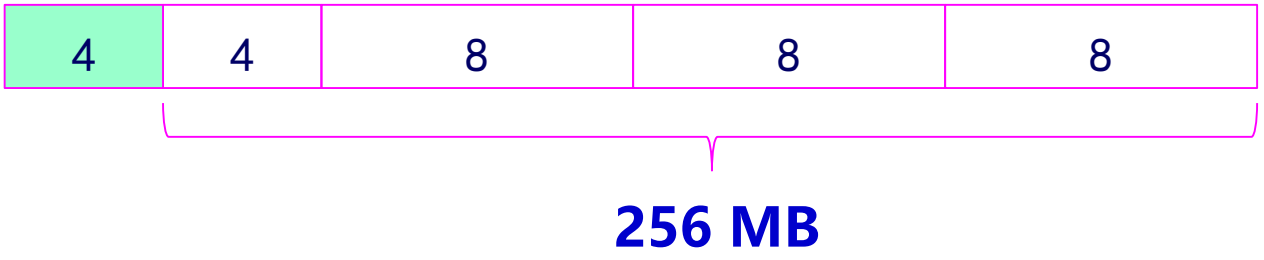
- **Memory Model**
  - The architecture can access up to 4 GBytes (address width is 32-bits) of unified program and I/O memory.
  - The address space is divided into 16 regions or segments [0H - FH], each of 256 MBytes.
  - The upper four bits of an address select the specific segment.



地址范围：0000 0000H~FFFF FFFFH
每个地址对应存储空间（内存）中的一个Byte
并不是每款单片机都有4GByte 内存空间

(2) 数据类型和寻址方式

- **Memory Model**
  - 注意：使用基地址base address+偏移地址offset访问，必须在一个Segment内！



Many data accesses use addresses computed by adding a displacement to the value of a base address register.

Using a displacement to cross one of the segment boundaries is not allowed and if attempted causes a MEM trap.

(2) 数据类型和寻址方式

- **Addressing Model寻址方式**
- The TriCore architecture supports seven addressing modes. The simple data elements are 8-bits, 16-bits, 32-bits and 64-bits wid
- These addressing modes support efficient compilation of C/C++ programs,
- Easy access to peripheral registers and efficient implementation of typical DSP data structures (circular buffers for filters and bit-reversed indexing for Fast Fourier Transformations)

- Addressing modes which are not directly supported in the hardware can be synthesized through short instruction sequences.

(2) 数据类型和寻址方式

● Addressing Modes 指令寻址方式

Table 2-4 Addressing Modes

Addressing Mode	Address Register Use
Absolute	None
Base + Short Offset	Address Register
Base + Long Offset	Address Register
Pre-increment	Address Register
Post-increment	Address Register
Circular	Address Register Pair
Bit-reverse	Address Register Pair

(2) 数据类型和寻址方式

- Addressing Modes 指令寻址方式
  - ◆Absolute addressing
    - ■ Absolute addressing is useful for referencing I/O peripheral registers and global data

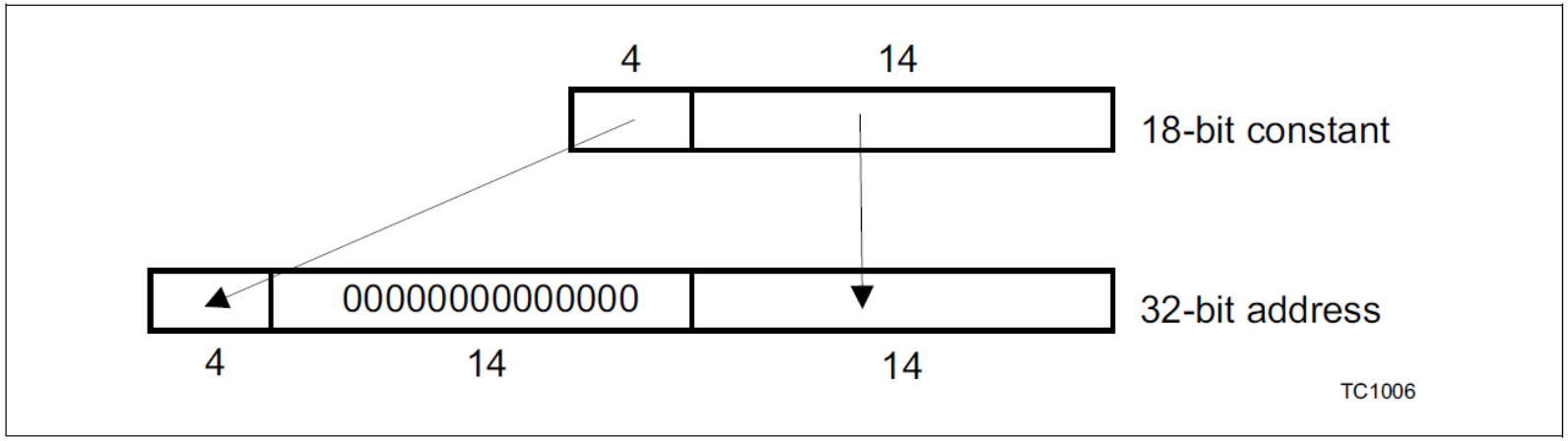
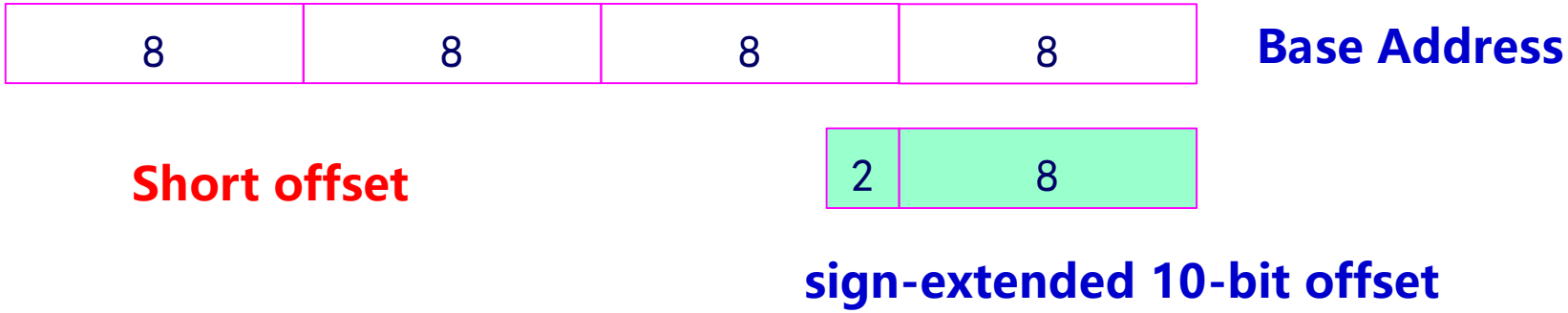


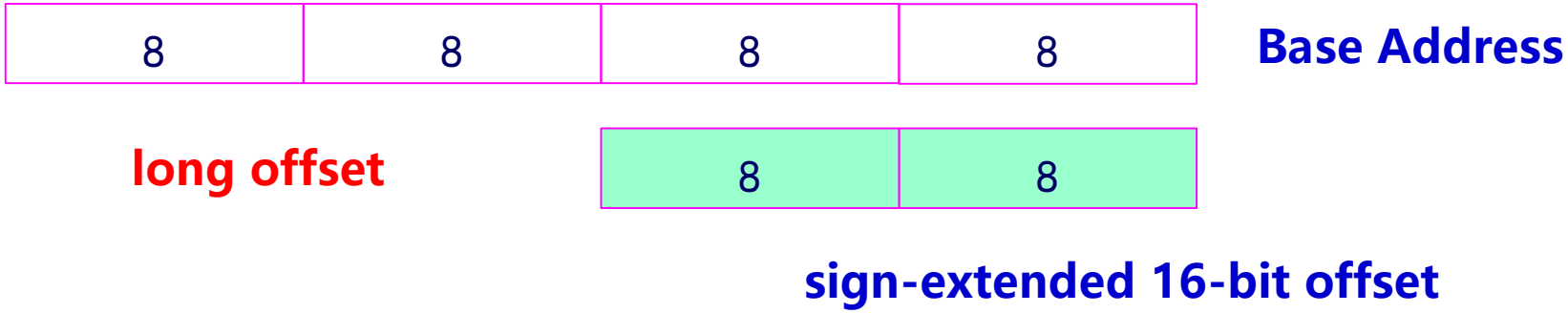
Figure 2-3 Translation of Absolute Address to Full Effective Address

注意：绝对地址访问并不能访问全部地址空间！

(2) 数据类型和寻址方式

- Addressing Modes 指令寻址方式
  - ◆Base + Offset Addressing



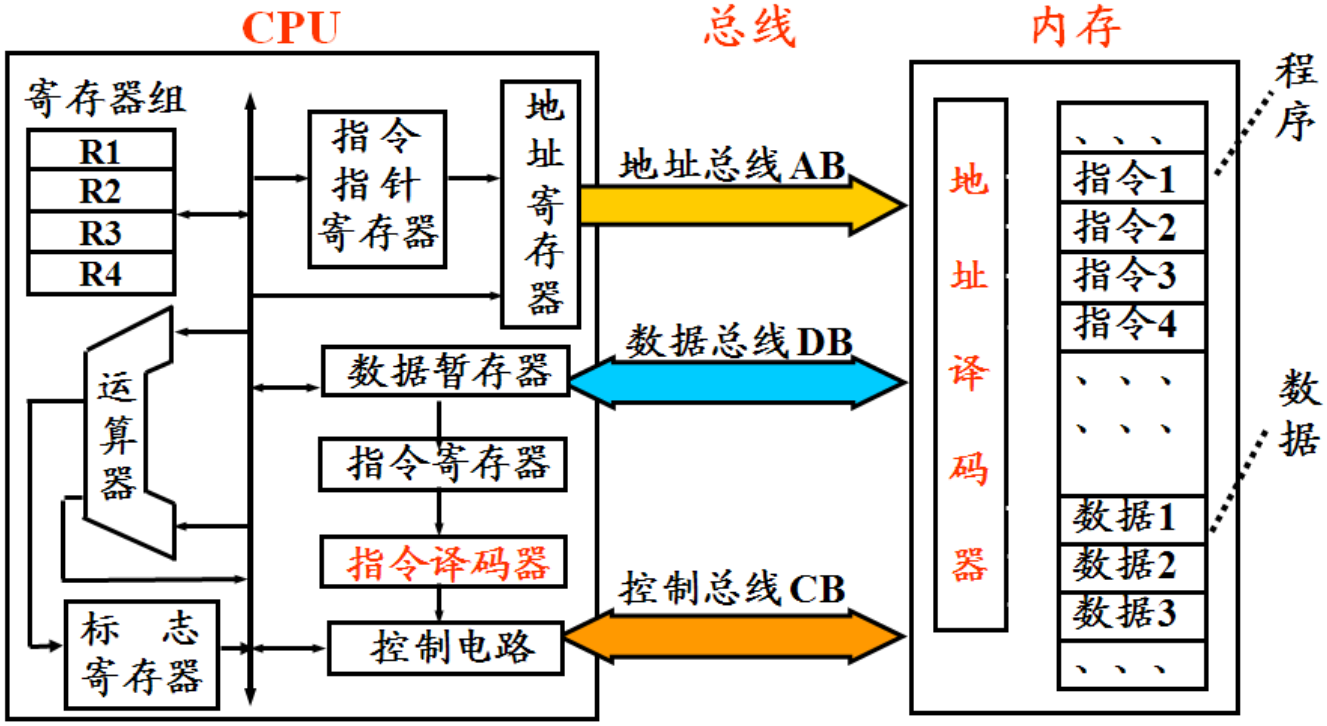


2条指令就可以自由访问全部地址空间：先给基地址寄存器赋值，再相对寻址

(2) 数据类型和寻址方式

- Addressing Modes 指令寻址方式
  - ◆Pre-Increment and Pre-Decrement Addressing
    - ■ 先加/减地址再寻址
  - ◆Post-Increment and Post-Decrement Addressing
    - ■先操作，再加/减寻址

(3) CPU系统寄存器



CPU示意图：以普林斯顿结构为例

基本要素：寄存器、任务/现场

(3) CPU系统寄存器

- TC2XX的CPU结构：Architectural Registers

The architectural registers consist of:

- 32 General Purpose Registers (GPRs)
- Program Counter (PC)
- Two 32-bit registers
  - PCXI - Previous ConteXt Information register,
  - PSW - Program Status Word

35个系统寄存器

(3) CPU系统寄存器

- TC2XX的CPU结构：Architectural Registers

35个系统寄存器，形成了单片机的程序现场context

- CPU运行不同任务，每个任务Task对应一个线程tread
- 每个线程在任何时刻都有一个现场context，现场是任务的时间切片
- 通过切换不同 现场，获得不同的线程，thread

- 8位单片机：CPU三个寄存器：堆栈指针、程序寄存器、数据寄存器
- 外围的周边模块，还有一堆寄存器
- 32位单片机，有几K个寄存器，每个寄存器有固定存储地址
- User's Manual会介绍寄存器，了解常用寄存器

(3) CPU系统寄存器

- TC2XX的CPU结构：Architectural Registers

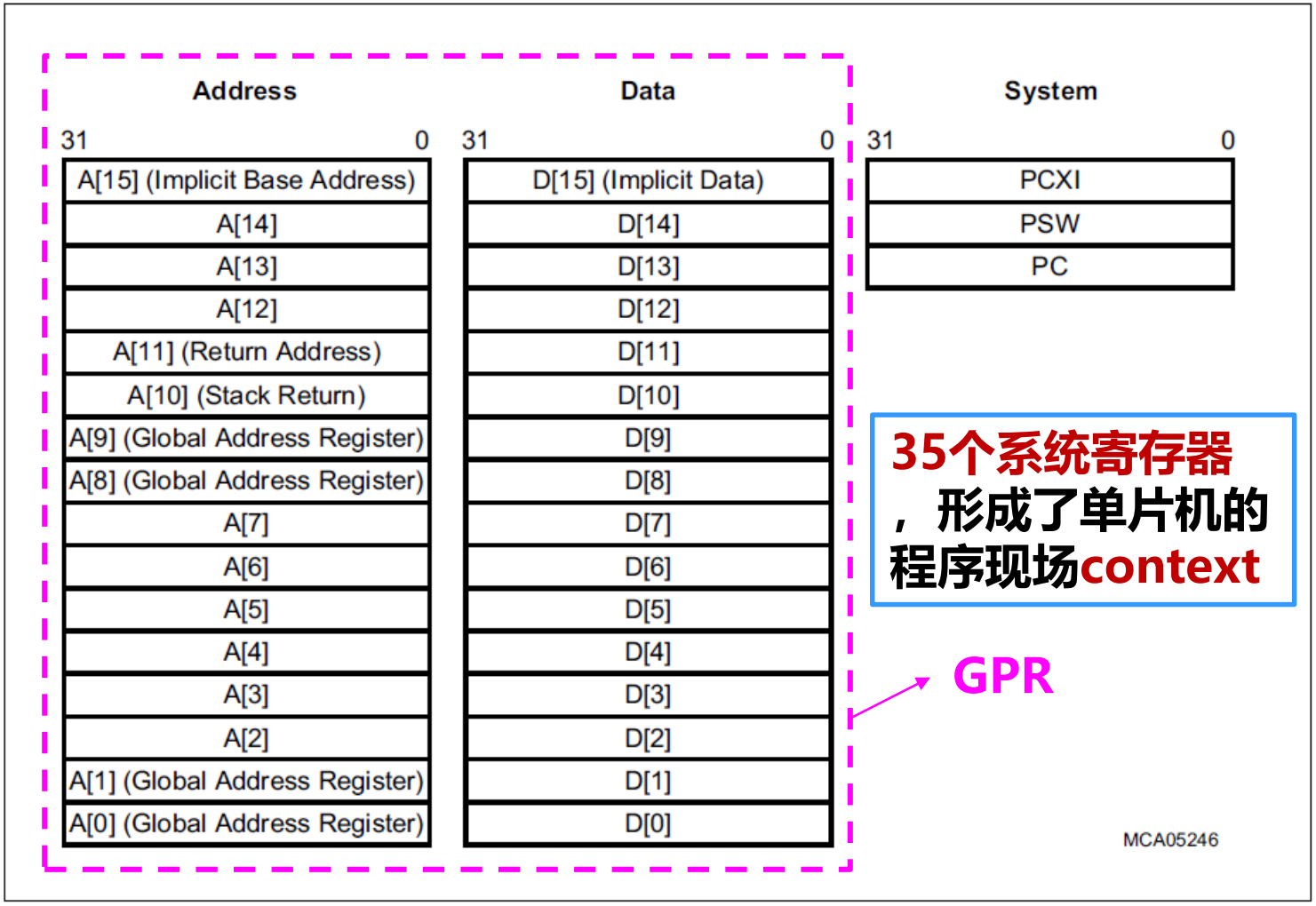


Figure 1-2 Architectural Registers

(3) CPU系统寄存器

- TC2XX的CPU结构：Architectural Registers

32个GPR：

D[0] ~ D[15]：数据寄存器

A[0] ~ A[15]：地址寄存器

其中包括4个特殊寄存器：

- D[15] is used as an Implicit Data register隐式数据寄存器
- A[10] is the Stack Pointer (SP) register堆栈寄存器
- A[11] is the Return Address (RA) register返回地址寄存器
- A[15] is the Implicit Address register隐式地址寄存器

编号 [0H - 7H] 叫‘lower registers’，编号 [8H - FH] are called the ‘upper registers’.

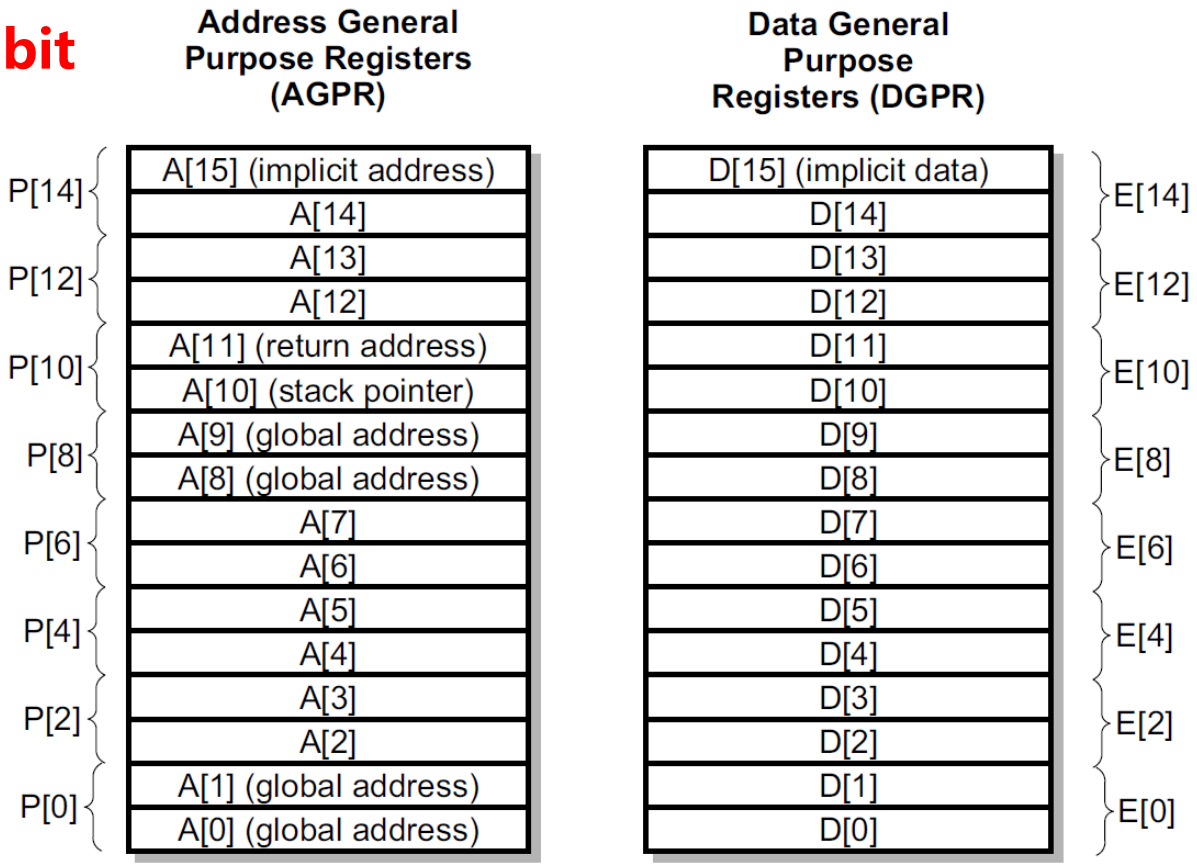
Registers A[0], A[1], A[8], and A[9] are defined as system global registers. 系统全局寄存器

(3) CPU系统寄存器

- General Purpose and System Registers

AGPRs： A[0]~A[15] ⟺ DGPRs： D[0]~D[15]

support 64-bit



(3) CPU系统寄存器

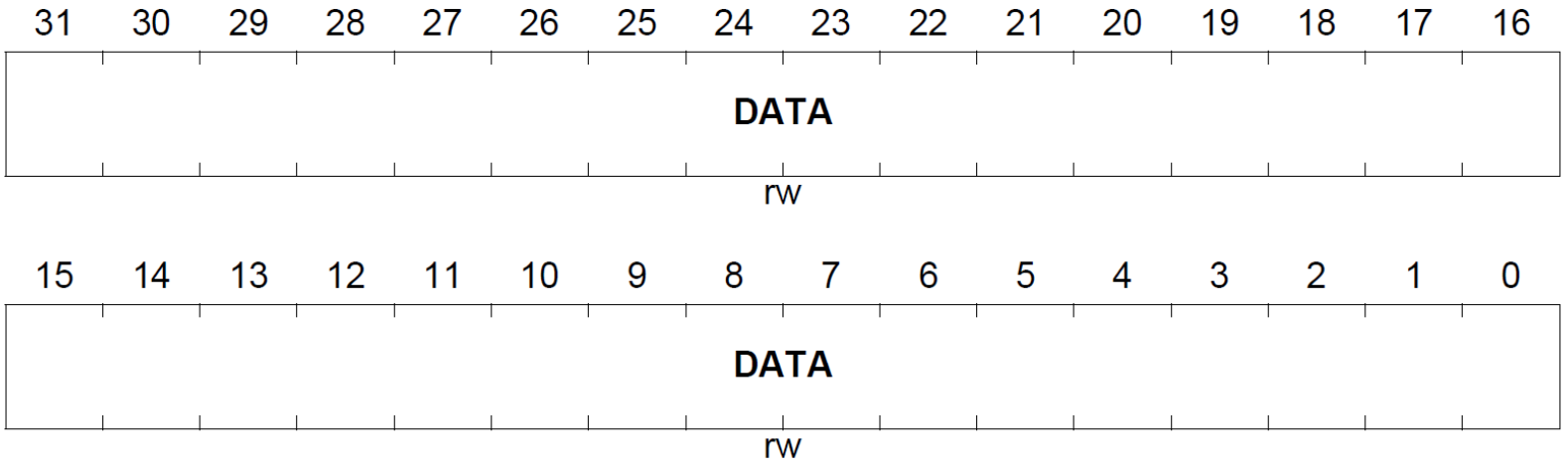
General Purpose and System Registers

Data General Purpose Registers

Dn (n=0-15)
Data Register n
Specific

($FF00_H$+n*4)

Reset Value: Implementation



Field	Bits	Type	Description
DATA	[31:0]	rw	Data Register n Value

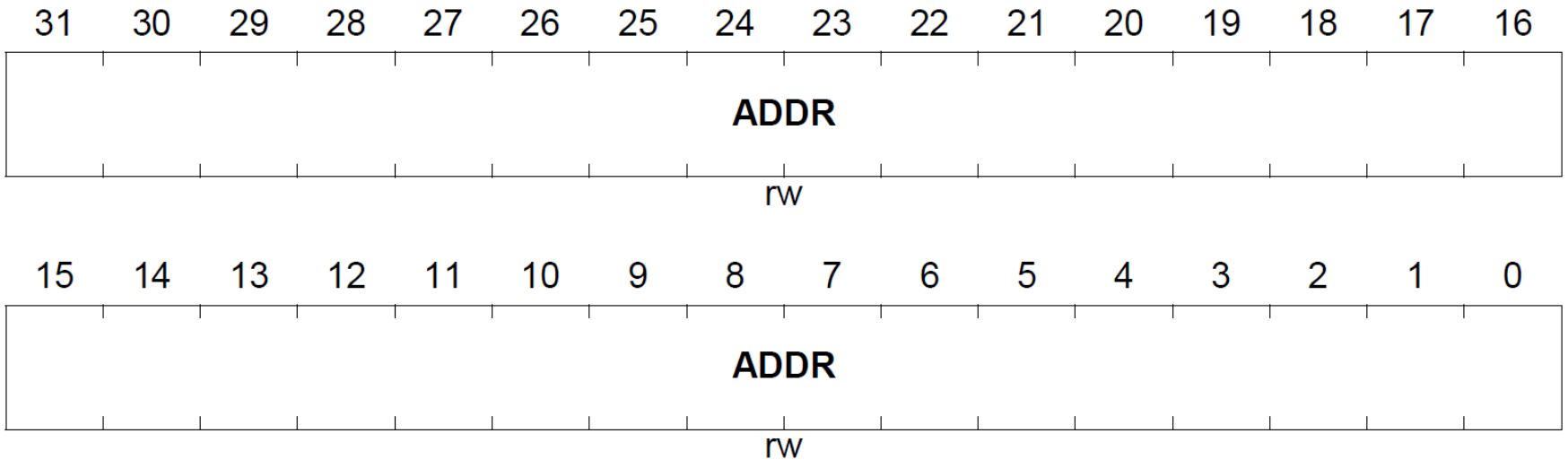
(3) CPU系统寄存器

General Purpose and System Registers

An (n=0-15)
Address Register n
Specific

($FF80_H$+n*4)

Reset Value: Implementation



Field	Bits	Type	Description
ADDR	[31:0]	rw	Address Register n Value

(3) CPU系统寄存器

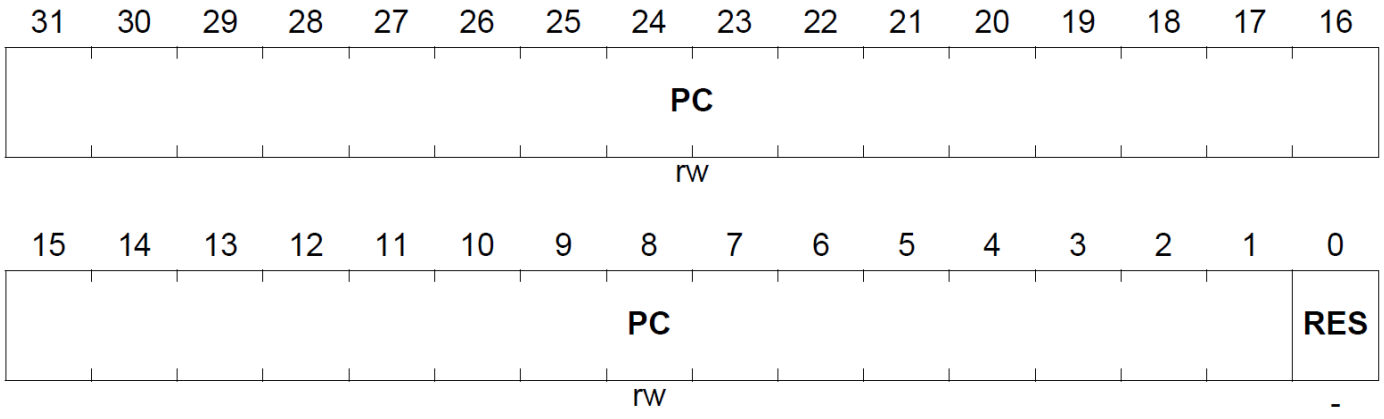
Program State Information Registers

- PC: Program Counter
- PSW: Program Status Word
- PCXI: Previous Context Information

PC
Program Counter Register
Specific

($FE08_H$)

Reset Value: Implementation



Field	Bits	Type	Description
PC	[31:1]	rw	Program Counter
RES	0	-	Reserved

(3) CPU系统寄存器

- **Program State Information Registers**
  - **PSW: Program Status Word**
    - **The Protection Register Set (PRS)**
    - **The I/O privilege level (IO)**
    - **The Interrupt Stack flag (IS)**
    - **The Global register Write permission flag (GW)**
    - **The Call Depth Counter (CDC)**
    - **The Call Depth Count Enable field (CDE)**

PSW
Program Status Word **($FE04_H$)** **Reset Value: 0000 0B80$_H$**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
USB								RES							
rw								-							

15	14	13	12	11-	10	9	8	7	6	5	4	3	2	1	0
RES	S	PRS		IO		IS	GW	CDE	CDC						
-	rw	rw		rw		rw	rw	rw	rw						

(3) CPU系统寄存器

- **Program State Information Registers**
  - **PSW: Program Status Word**

PSW User Status Bits

Field	Bits	Type	Description
C	31	rw	Carry
V	30	rw	Overflow
SV	29	rw	Sticky Overflow
AV	28	rw	Advance Overflow
SAV	27	rw	Sticky Advance Overflow
RES	[26:24]	-	Reserved Field

执行carry
溢出overflow

(3) CPU系统寄存器

Program State Information Registers

- **PCXI : Previous Context Information and Pointer Register**

PCXI. PCX
Previous Context Information and Pointer Register

($FE00_H$)

Reset Value: Implementation Specific

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RES		PCPN								PIE	UL	PCXS			
-		rw								rw	rw	rw			

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PCXO															
rw															

(3) CPU系统寄存器

Program State Information Registers

- **PCXI : Previous Context Information and Pointer Register**

Field	Bits	Type	Description
RES	[31:30]	-	**Reserved**
PCPN	[29:22]	rw	**Previous CPU Priority Number** Contains the priority level number of the interrupted task.
PIE	21	rw	**Previous Interrupt Enable** Indicates the state of the interrupt enable bit (ICR.IE) for the interrupted task.
UL	20	rw	**Upper or Lower Context Tag** Identifies the type of context saved: 0 : Lower Context 1 : Upper Context If the type does not match the type expected when a context restore operation is performed, a trap is generated.
PCXS	[19:16]	rw	**PCX Segment Address** Contains the segment address portion of the PCX. This field is used in conjunction with the PCXO field.
PCXO	[15:0]	rw	**Previous Context Pointer Offset Field** The PCXO and PCXS fields form the pointer PCX, which points to the CSA of the previous context.

(4) 任务与现场

- Tasks and Contexts 任务与现场
  - A task is an independent thread （线程）of control. There are two types: Software Managed Tasks (SMTs) and Interrupt Service Routines (ISRs).
  - SMTs are created through the services of a real-time kernel or Operating System, and are dispatched under the control of scheduling software. ISRs are dispatched by hardware in response to an interrupt.
  - An ISR is the code that is invoked directly by the processor on receipt of an interrupt. SMTs are sometimes referred to as user tasks, assuming that they execute in User Mode.

基本概念：线程thread和任务task，SMT，ISR
关键词：real-time kennel, operating system, software, hardware, interrupt

(4) 任务与现场

- Tasks and Contexts 任务与现场
  - ◆ Each task is allocated its own mode, depending on the task's function:

• Supervisor Mode: Permits read/write access to system registers and all peripheral devices. Tasks in this mode may disable interrupts.

• User Mode 1

• User Mode 0

Individual modes are enabled or disabled primarily through the I/O mode bits in the Processor Status Word (PSW).

基本概念：任务运行模式mode

(4) 任务与现场

- Tasks and Contexts 任务与现场
  - A set of state elements are associated with any task, and these are known collectively as the task's context.
  - The context is everything the processor needs to define the state of the associated task and enable its continued execution.
  - This includes the CPU General Registers that the task uses, the task's Program Counter (PC), and its Program Status Information (PCXI and PSW). The architecture efficiently manages and maintains the context of the task through hardware.
  - The context is subdivided into the upper context and the lower context.

基本概念：任务现场context

(4) 任务与现场

- Tasks and Contexts 任务与现场
  - Context Save Areas (CSAs, 现场保存区)
  - A CSA consists of 16 words of memory storage, aligned on a 16-word boundary.
  - Each CSA can hold exactly one upper or one lower context. CSAs are linked together through a Link Word.
  - The architecture saves and restores context more quickly than conventional microprocessors and microcontrollers

基本概念：现场保存区CSA、链接字Link Word
35个寄存器（每个长度为1个word=32bit），怎么用16word的CAS存储?

(4) 任务与现场

案例分析

- 燃料电池空压机额定转速每分钟15万转
- 15万/60s-2500转/s
- 1s, 空压机电机控制器至少产生2500个正弦信号，2500Hz
- 工程上，一个正弦波至少10次采样/控制才能得到较号的控制效果
- 2500*10=25kHz
- 即：空压机电机控制器的IGBT必须以25kHz的频率开关动作才可以实现空压机稳定高速旋转
- 此时，控制IGBT开关的中断服务程序频率25kHz，对应时间周期1/25kHz=40us

- 一个中断程序运行时间<40微秒，时间要求极其苛刻
- CSA用16个寄存器存取32个寄存器有效信息，速度提升2倍，意义重大

(4) 任务与现场

- Tasks and Contexts 任务与现场
  - Context Save Areas (CSAs)
  - Context switching occurs when an event or instruction causes a break in program execution. The CPU then needs to resolve this event before continuing with the program.
  - The events and instructions which cause a break in program execution are:
    - Interrupt（中断） or service requests（服务请求）
    - Traps（陷阱）
    - Function calls（函数调用）

何时发生现场保存和切换?
Interrupt, service request, trap, function call

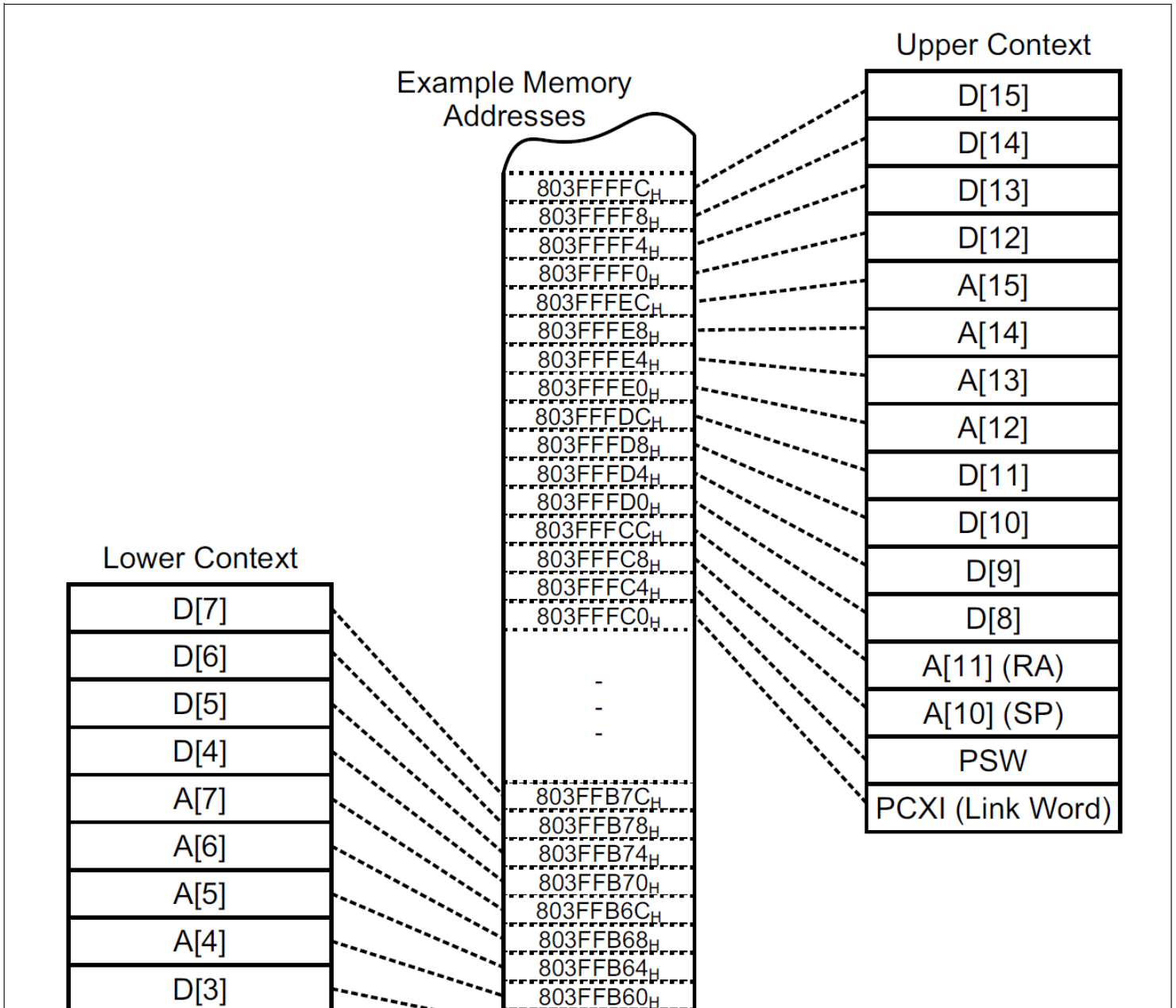
(4) 任务与现场

Context Types

- A task is an independent thread of control. The state of a task is defined by its context.
- When a task is interrupted, the processor uses that task's context to re-enable the continued execution of the task.
- Upper context: Consists of the upper address registers A[10] to A[15] and the upper data registers D[8] to D[15]. (also includes PCXI and PSW)
- Lower context: Consists of the lower address registers A[2] to A[7], the lower data registers D[0] to D[7], A[11] (Return Address) and PCXI

现场怎么保护？在RAM里面开辟缓冲区
分成高半现场和低半现场
高半现场和低半现场（16word）的组成?
A[0\1\8\9]、PC?

(4) 任务与现场


Upper Context
D[15]
D[14]
D[13]
D[12]
A[15]
A[14]
A[13]
A[12]
D[11]
D[10]
D[9]
D[8]
A[11] (RA)
A[10] (SP)
PSW
PCXI (Link Word)
Example Memory Addresses
803FFFFC$_H$
803FFFF8$_H$
803FFFF4$_H$
803FFFF0$_H$
803FFFEC$_H$
803FFFE8$_H$
803FFFE4$_H$
803FFFE0$_H$
803FFFDC$_H$
803FFFD8$_H$
803FFFD4$_H$
803FFFD0$_H$
803FFFCC$_H$
803FFFC8$_H$
803FFFC4$_H$
803FFFC0$_H$
803FFB7C$_H$
803FFB78$_H$
803FFB74$_H$
803FFB70$_H$
803FFB6C$_H$
803FFB68$_H$
803FFB64$_H$
803FFB60$_H$
Lower Context
D[7]
D[6]
D[5]
D[4]
A[7]
A[6]
A[5]
A[4]
D[3]

Context Types--Context Save Area

CSAs are linked together through a Link Word

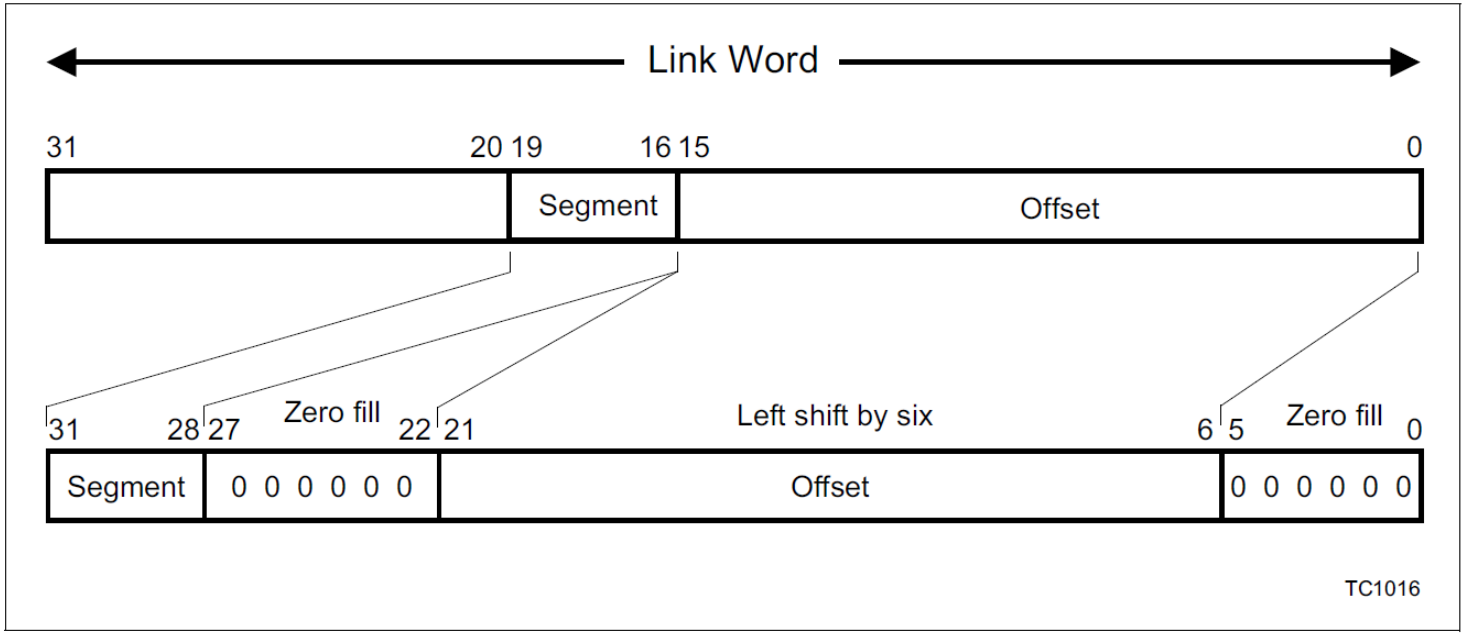


Figure 4-2 Generation of the Effective Address of a Context Save Area (CSA)

链接字Link Word：PCXI.PCX

(4) 任务与现场

- **Context Types**
  - ◆ After the context save operation has been performed the Return Address A[11](RA) is updated:
  - For a call, the A[11](RA) is updated with the function return address.
  - For a synchronous trap, the A[11](RA) is updated with the PC of the instruction which raised the trap.
  - For a SYSCALL and an asynchronous trap or an interrupt, the A[11](RA) is updated with the PC of the next instruction to be executed.

  Both lower context and upper context save A[11](RA).

- **Fast Function Calls with FCALL/FRET**

  Only save A[11](RA) register! 快速调用和快速返回。不影响其他寄存器的内容!

(4) 任务与现场

Context Types--Context Save Area

- The upper context is saved automatically as a result of an external interrupt, trap or function call. 高半现场硬件自动保存，也可以选择用软件来保存!
- The lower context is saved explicitly through instructions. 低半现场，只有靠软件保存!

- The lower context registers are similar to global registers in the sense that a interrupt handler, trap handler or called function, sees the same values that were present in the registers just before the interrupt, trap or call.
- 用指令Return From Call (RET)或者Return From Exception (RFE) 返回中断服务程序中的lower context的值。

低半现场不保存，等价为全局寄存器（Global Register）
可用来做什么？传递重要的参数和返回结果!

(4) 任务与现场

- Context Types--Context Save Area

延伸：加快软件代码运行的底层逻辑方法

（1）设计代码内存管理方式，只调用高半现场寄存器

（2）Fast Call, Fast Return，只保存A[11]（返回地址）

另：进入陷阱Trap，CPU也会自动硬件保存高半现场，运行REF指令返回原线程

Context Types--Context Save Area

Table 4-1 Context Related Events and Instructions

Event / Instruction	Context Operation	Complement Instruction	Context Operation
Interrupt	Save Upper	RFE - Return from Exception	Restore Upper
Trap	Save Upper	RFE - Return from Exception	Restore Upper
CALL - Function Call	Save Upper	RET - Return from Call	Restore Upper
BISR - Begin Interrupt Service Routine	Save Lower	RSLCX - Restore Lower Context	Restore Lower
SVLCX - Save Lower Context	Save Lower	RSLCX - Restore Lower Context	Restore Lower
STLCX - Store Lower Context	Store Lower	LDLCX - Load Lower Context	Load Lower
STUCX - Store Upper Context	Store Upper	LDUCX - Load Upper Context	Load Upper

Event事件、Instruction指令

- Context Types
  - ◆Context Save Areas (CSAs) and Context Lists

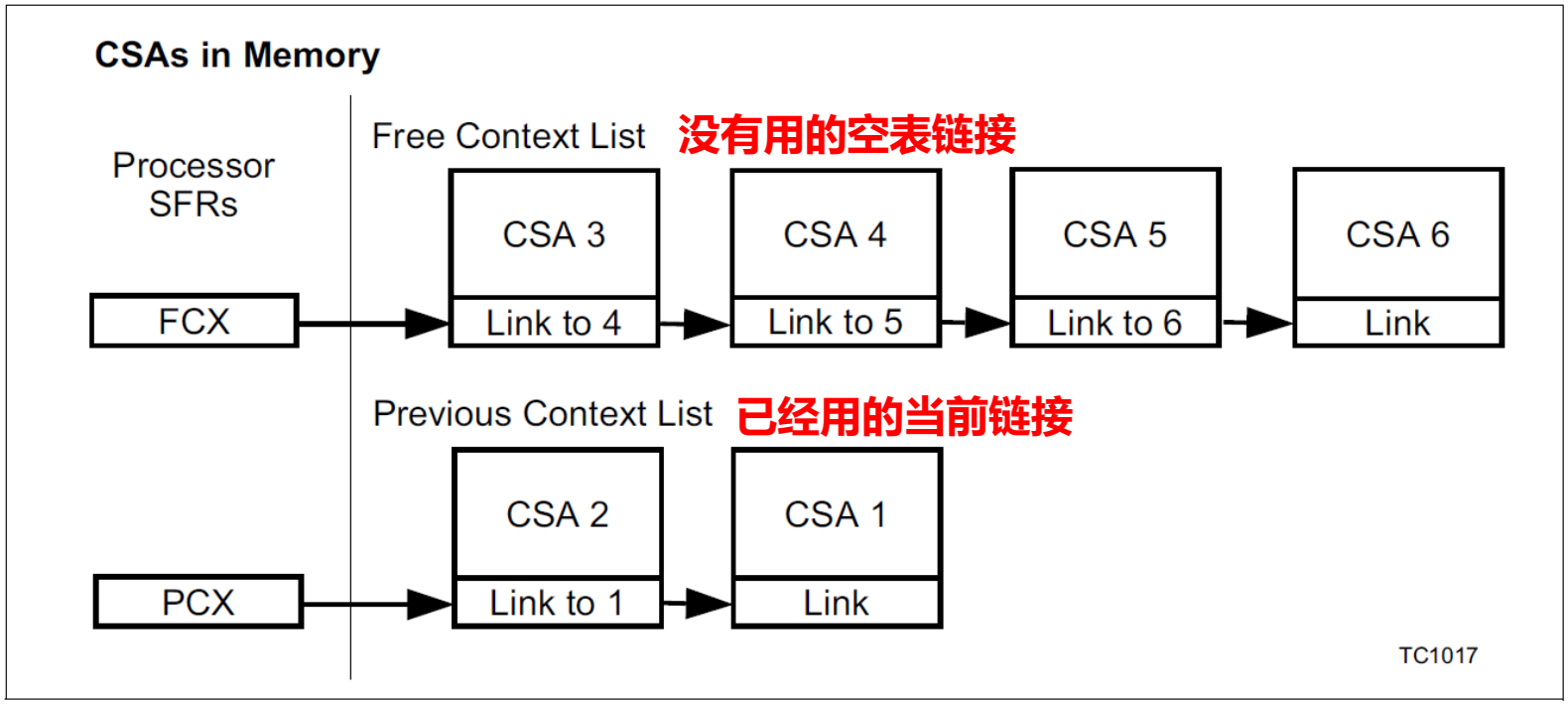


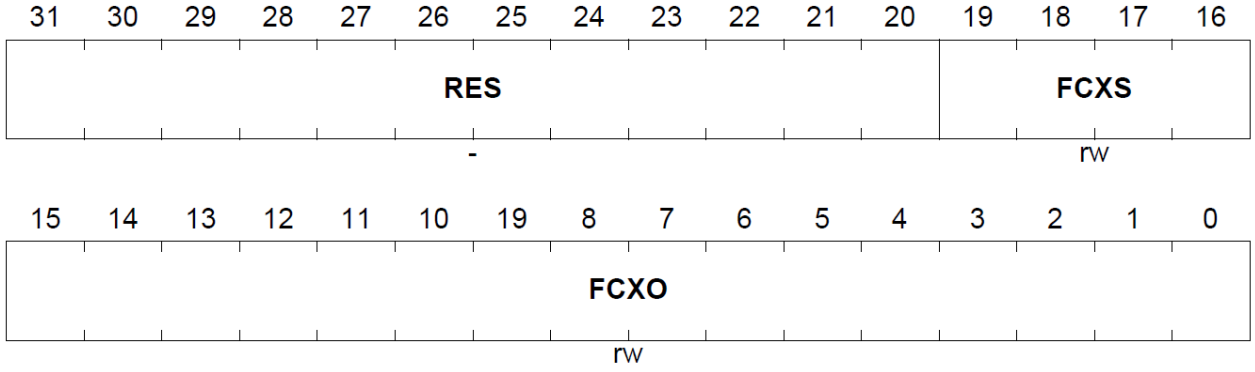
Figure 4-3 CSAs in Context Lists

Context Management Registers

◆Free CSA List Head Pointer Register (FCX)

FCX
Free CSA List Head Pointer **($FE38_H$)**

Reset Value: Implementation Specific



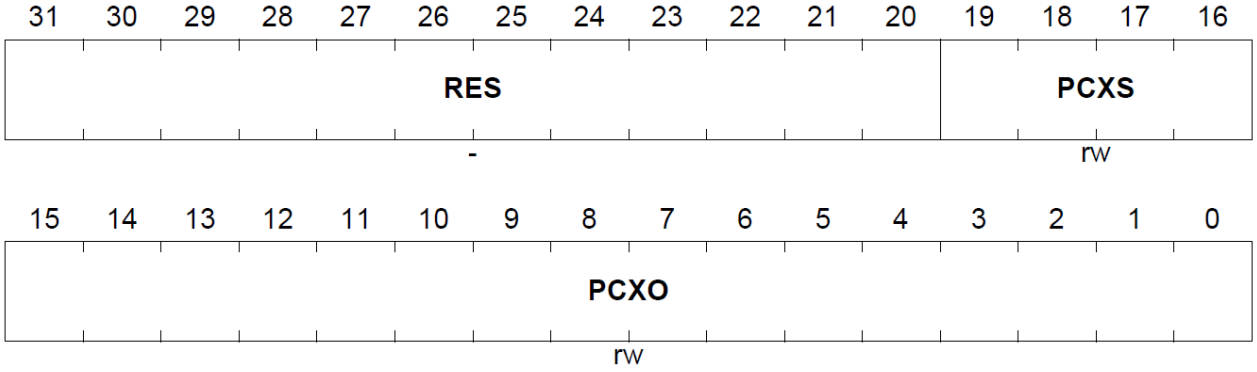
Field	Bits	Type	Description
RES	[31:20]	-	**Reserved**
FCXS	[19:16]	rw	**FCX Segment Address** Used in conjunction with the FCXO field.
FCXO	[15:0]	rw	**FCX Offset Address** The FCXO and FCXS fields together form the FCX pointer, which points to the next available CSA.

Context Management Registers

◆Previous Context Pointer Register (PCX)

PCX
Previous Context Pointer Register $(FE00_H)$

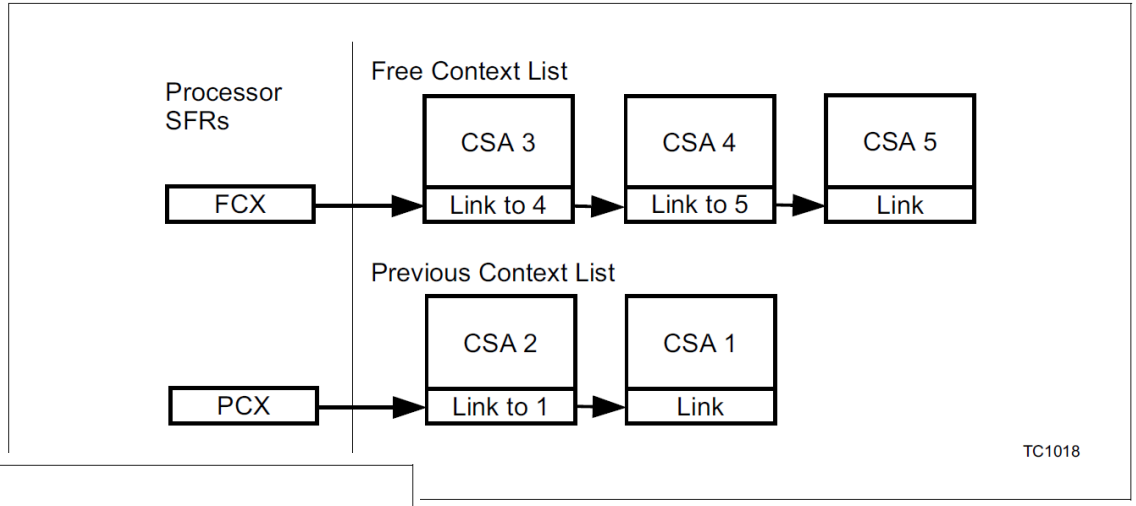
Reset Value: Implementation Specific

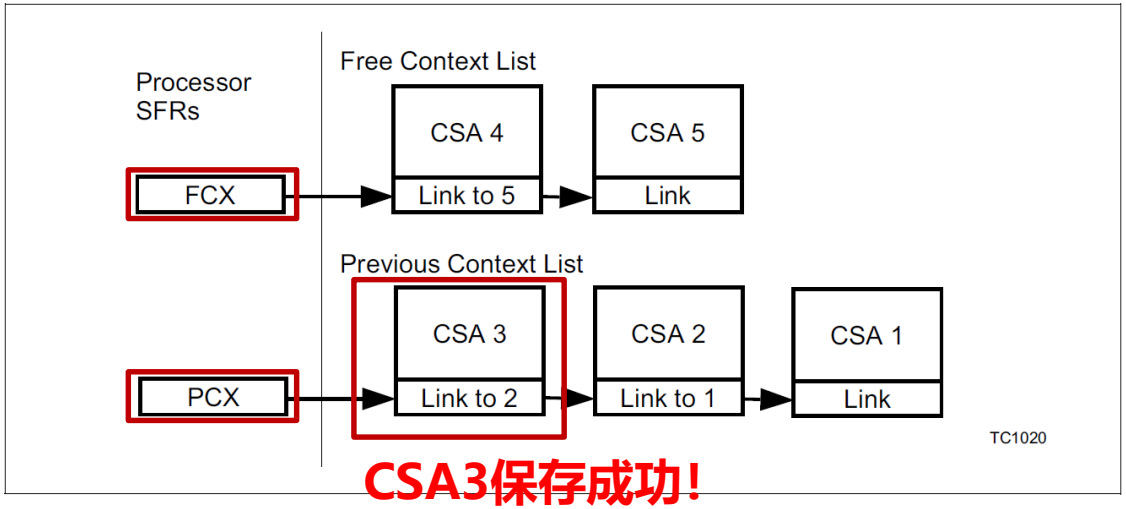


Field	Bits	Type	Description
RES	[31:20]	-	**Reserved**
PCXS	[19:16]	rw	**Previous Context Pointer Segment Address** This field is used in conjunction with the PCXO field.
PCXO	[15:0]	rw	**Previous Context Pointer Offset** The PCXO and PCXS fields form the pointer PCX, which points to the CSA of the previous context.

(4) 任务与现场

- Context Save and Restore
  - ◆如何保存一条Context





CSA3保存成功!

Context Save and Restore

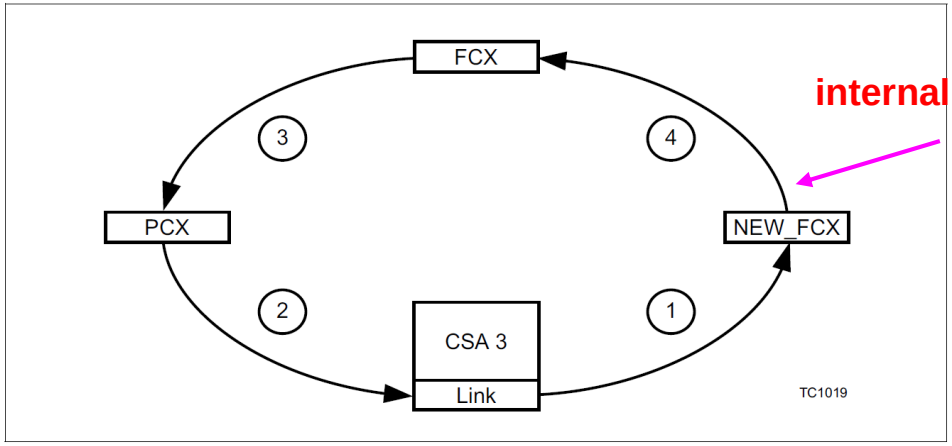
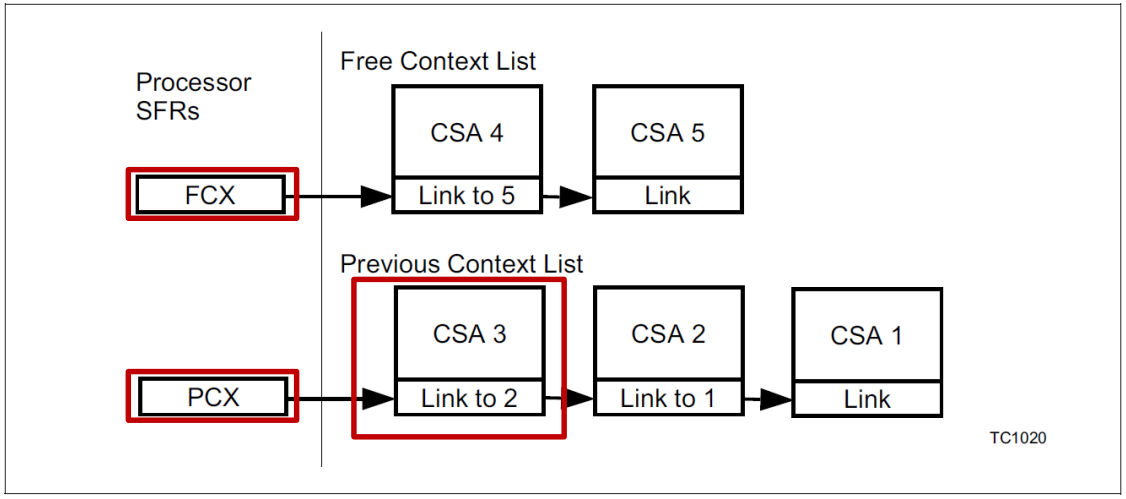


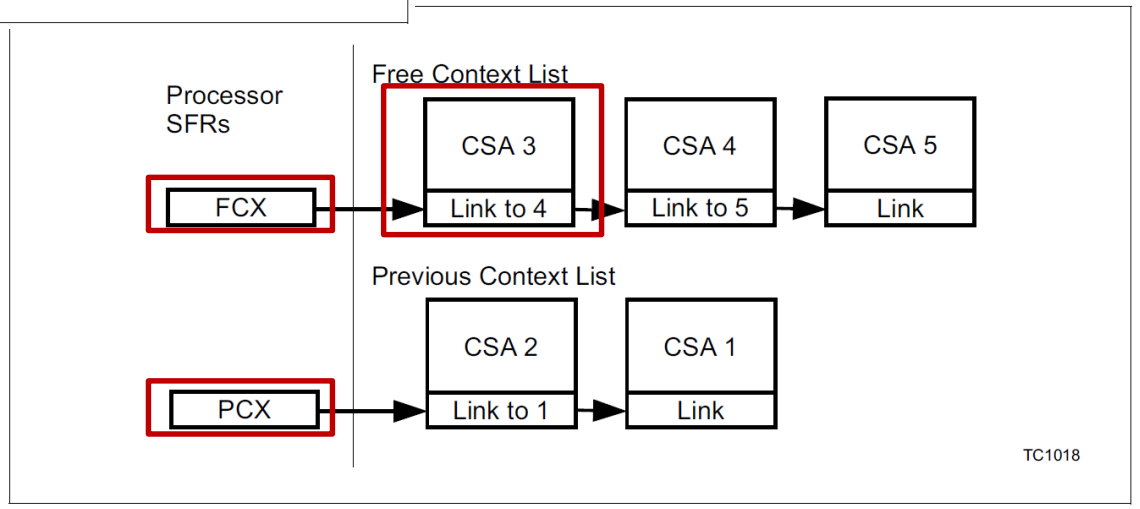
Figure 4-5 CSA and Processor SFR Updates on a Context Save Process

1. The contents of the Link Word in CSA3 are loaded into the NEW_FCX. The NEW_FCX now points to CSA4. The NEW_FCX is an internal buffer and is not accessible by the user.
2. The contents of the PCX are written into the Link Word of CSA3. The Link Word of CSA3 now points to CSA2.
3. The contents of FCX are written into the PCX. The PCX now points to CSA3, which is at the front of the Previous Context List.
4. The NEW_FCX is loaded into the FCX.

(4) 任务与现场

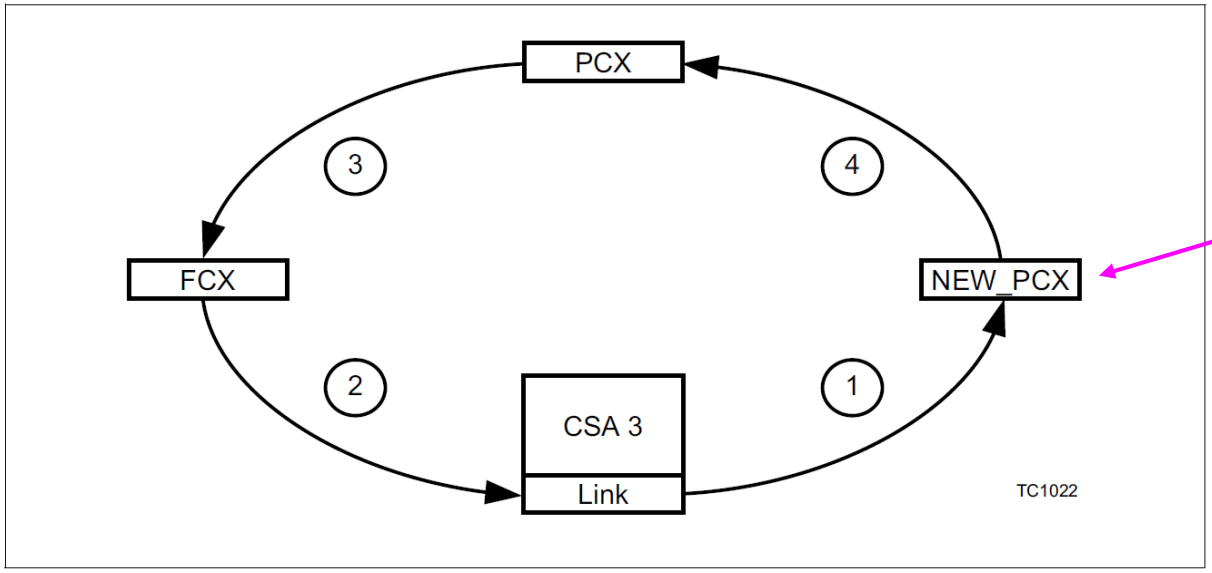
- Context Save and Restore
  - 如何取出一条Context，比如把CSA3取出?





(4) 任务与现场

- Context Save and Restore Examples
  - 如何取出一条Context，比如把CSA3取出?



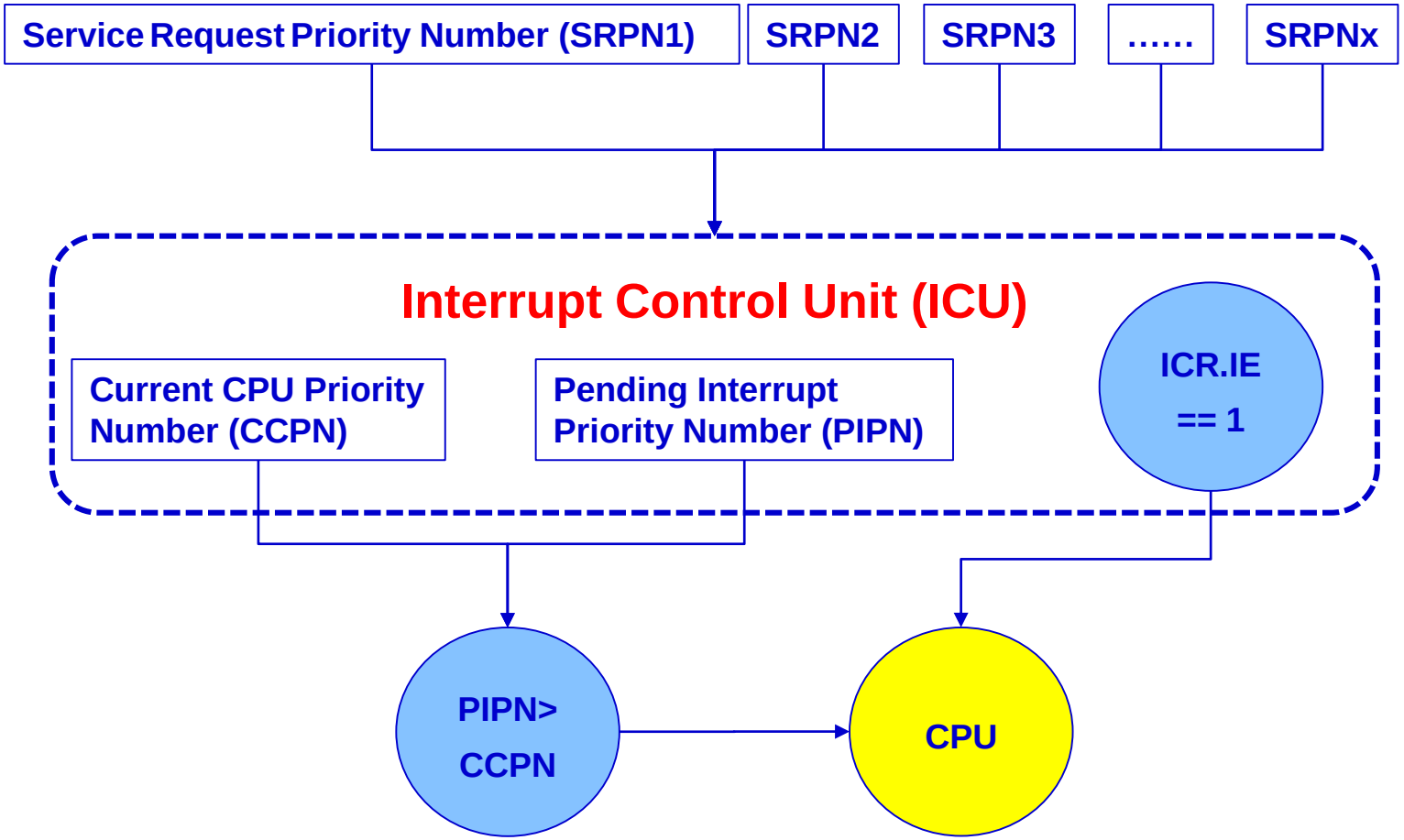
1. The contents of the Link Word in CSA3 are loaded into the NEW_PCX. The NEW_PCX now points to CSA2. The NEW_PCX is an internal buffer and is not accessible by the user.
2. The contents of the FCX are written into the Link Word of CSA3. The Link Word of CSA3 now points to CSA4.
3. The contents of the PCX are written into the FCX. The FCX now points to CSA3, which is at the front of the free context list.
4. The NEW_PCX is loaded into the PCX.

(5) Interrupt System中断系统

- **Interrupt System**
  - A key feature of the architecture is its powerful and flexible interrupt system. The interrupt system is built around programmable Service Request Nodes (SRNs，服务请求节点).
  - A Service Request is defined as an interrupt request or a DMA (Direct Memory Access) request. A service request may come from an on-chip peripheral, external hardware, or software.

(5) Interrupt System中断系统

- 中断过程



(5) Interrupt System中断系统

- Interrupt Priority
  - ◆中断控制单元（Interrupt Control Unit，ICU）决定哪个中断源赢得中断The Interrupt Control Unit (ICU) determines which source will win arbitration based on the priority number
  - ◆每个SR分配优先级Priority Number（SRPNs）
  - ◆每个中断程序(Interrupt Service Routine, ISR)都有自己的优先级数，不同的服务请求（Service Request, SR）必须有不同的优先级数
  - ◆CPU最多有255个中断源，最多可设255个中断优先级
  - ◆中断嵌套：优先级高的SR可以中断优先级低的SR（嵌套）；同级的SR不能彼此中断

(5) Interrupt System中断系统

ICU Interrupt Control Register (ICR)

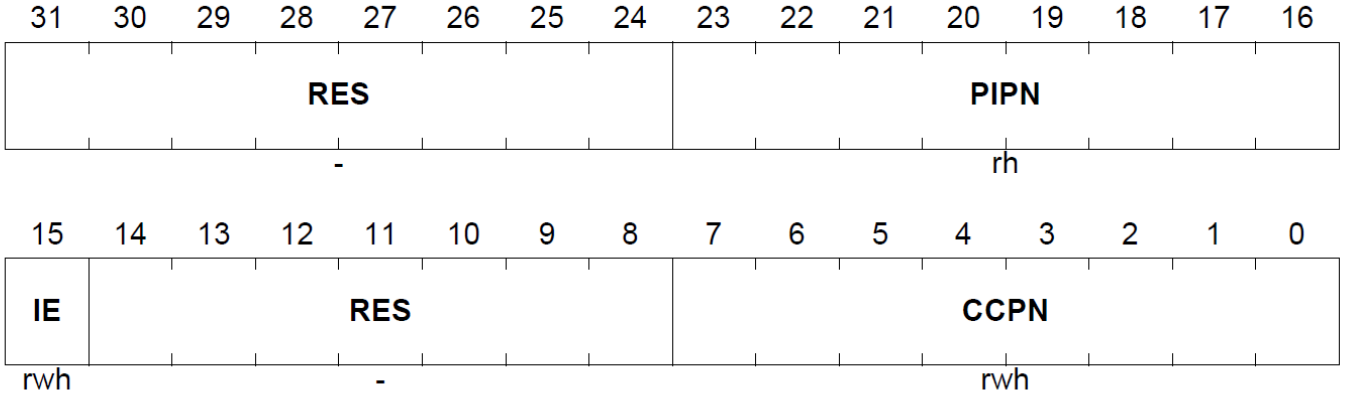
- The ICU Interrupt Control Register (ICR) holds the Current CPU Priority Number (CCPN), the global Interrupt enable/disable bit (IE) and the current Pending Interrupt Priority Number (PIPN).

Several conditions could block the CPU from immediately responding to the interrupt request generated by the ICU.

- The interrupt system is globally disabled (ICR.IE == 0).
- The current CPU priority (CCPN), is equal to or higher than the Pending Interrupt Priority Number (PIPN).
- The CPU is in the process of entering an interrupt or trap service routine.
- The CPU is operating on non-interruptible trap services.
- The CPU is executing a multi-cycle instruction.
- The CPU is executing an instruction which modifies the ICR.

(5) Interrupt System中断系统

ICR
ICU Interrupt Control **($FE2C_H$)** **Reset Value: $0000\ 0000_H$**



Field	Bits	Type	Function
RES	[31:24]	-	**Reserved**
PIPN	[23:16]	rh	**Pending Interrupt Priority Number** A read-only bit field that is updated by the ICU at the end of each interrupt arbitration process. It indicates the priority number of the pending service request. ICR.PIPN is set to 0 when no request is pending, and at the beginning of each new arbitration process. 00_H : No valid pending request. 01_H : Request pending, lowest priority. … FF_H : Request pending, highest priority.

(5) Interrupt System中断系统

Field	Bits	Type	Function
IE	15	rwh	**Global Interrupt Enable Bit** The interrupt enable bit globally enables the CPU service request system. Whether a service request is delivered to the CPU depends on the individual Service Request Enable Bits (SRE) in the SRNs, and the current state of the CPU. ICR.IE is automatically updated by hardware on entry and exit of an Interrupt Service Routine (ISR). ICR.IE is cleared to 0 when an interrupt is taken, and is restored to the previous value when the ISR executes an RFE instruction to terminate itself. ICR.IE can also be updated through the execution of the ENABLE, DISABLE, MTCR, and BISR instructions. 0 : Interrupt system is globally disabled. 1 : Interrupt system is globally enabled.
RES	[14:8]	-	**Reserved Field**
CCPN	[7:0]	rwh	**Current CPU Priority Number** The Current CPU Priority Number (CCPN) bit field indicates the current priority level of the CPU. It is automatically updated by hardware on entry or exit of Interrupt Service Routines (ISRs) and through the execution of a BISR instruction. CCPN can also be updated through an MTCR instruction.

(5) Interrupt System中断系统

- 如何进中断Entering an Interrupt Service Routine (ISR)

1) The upper context of the current task is saved.
2) The Return Address (A[11]) is updated with the current PC.
3) If the processor was not previously using the interrupt stack (PSW.IS = 0), then the A[10] Stack Pointer is set to the interrupt stack pointer (ISP). The stack pointer bit is then set for using the interrupt stack: PSW.IS = 1.
4) The I/O mode is set to Supervisor mode, which means all permissions are enabled: PSW.IO = 10B.
5) The current Protection Register Set is set to 0: PSW.PRS = 00B.
6) The Call Depth Counter (PSW.CDC) is cleared, and the call depth limit selector is set for 64: PSW.CDC = 0000000B.
7) Call Depth Counter is enabled, PSW.CDE = 1.
8) PSW Safety bit is set to value defined in the SYSCON register. PSW.S = SYSCON.IS.
9) Write permission to global registers A[0], A[1], A[8], A[9] is disabled: PSW.GW = 0.
10) The interrupt system is globally disabled: ICR.IE = 0. The old ICR.IE is saved into PCXI.PIE.

(5) Interrupt System中断系统

- 如何进中断Entering an Interrupt Service Routine (ISR)

11) The Current CPU Priority Number (ICR.CCPN) is saved into the Previous CPU Priority Number (PCXI.PCPN) field.

12) The Pending Interrupt Priority Number (ICR.PIPN) is saved into the Current CPU Priority Number (ICR.CCPN) field.

13) The interrupt vector table is accessed to fetch the first instruction of the ISR.

ENABLE指令：ICR.IE =1， DISABLE指令ICR.IE =0

Note: Global register write permission is disabled (PSW.GW == 0) whenever an Interrupt Service Routine or trap handler is entered. This ensures that all traps and interrupts must assume they do not have write access to the registers controlled by PSW.GW by default.

用RFE (Return From Exception)指令退出中断服务程序

(5) Interrupt System中断系统

- Interrupt System
  - ◆Conventional architectures generally take a long time to service interrupt requests, and they are normally handled by loading a new Program Status (PS) from a vector table in data memory.
  - ◆In the TriCore architecture, service requests jump to vectors in code memory to reduce response time.
  - ◆The entry code for the ISR is a block within a vector of code blocks.
  - ◆ Each code block provides an entry for one interrupt source.

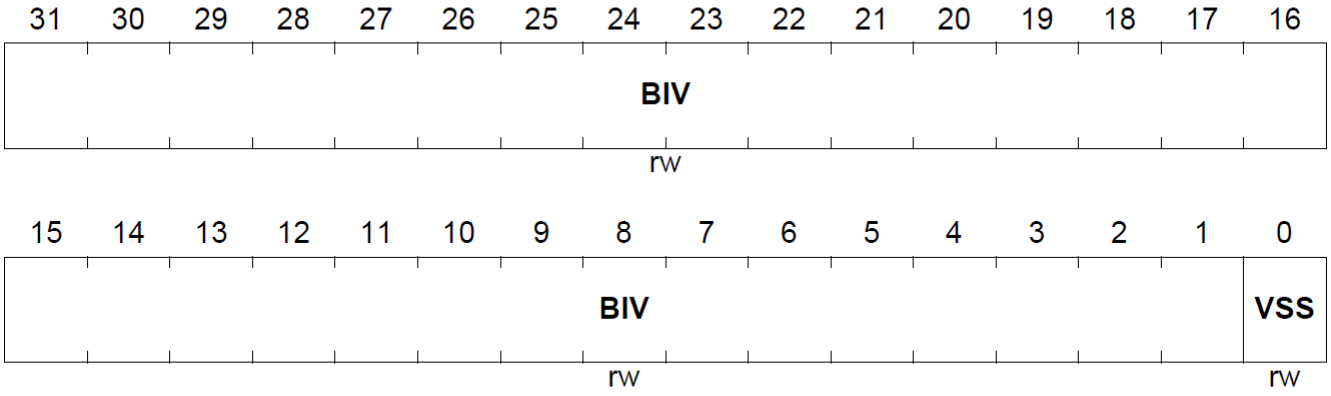
(5) Interrupt System中断系统

- Interrupt Vector Table
  - priority of the interrupt (the ICR.PIPN bit field)
  - Interrupt Vector Table → the start of the selected Interrupt Service Routine (ISR)
  - The Base of Interrupt Vector Table register (BIV) → ENDINIT protected
  - To generate a pointer into the Interrupt vector table the PIPN is left-shifted by either five bits (VSS=0), or three bits (VSS=1) and ORed with the address in the BIV register to generate a pointer into the Interrupt Vector Table.

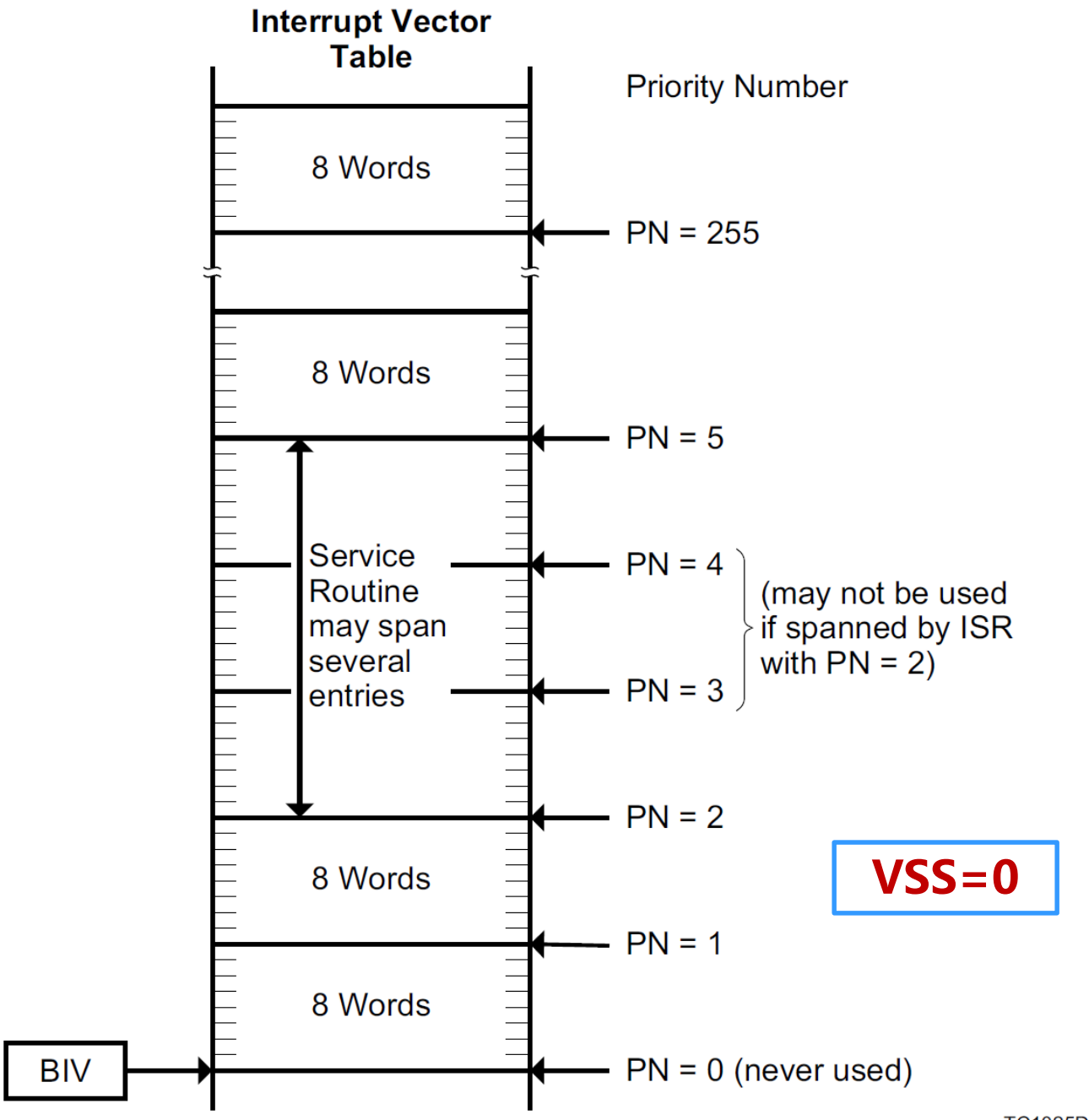
(5) Interrupt System中断系统

BIV
Base Interrupt Vector Table Pointer ($FE20_H$)

Reset Value: Implementation Specific

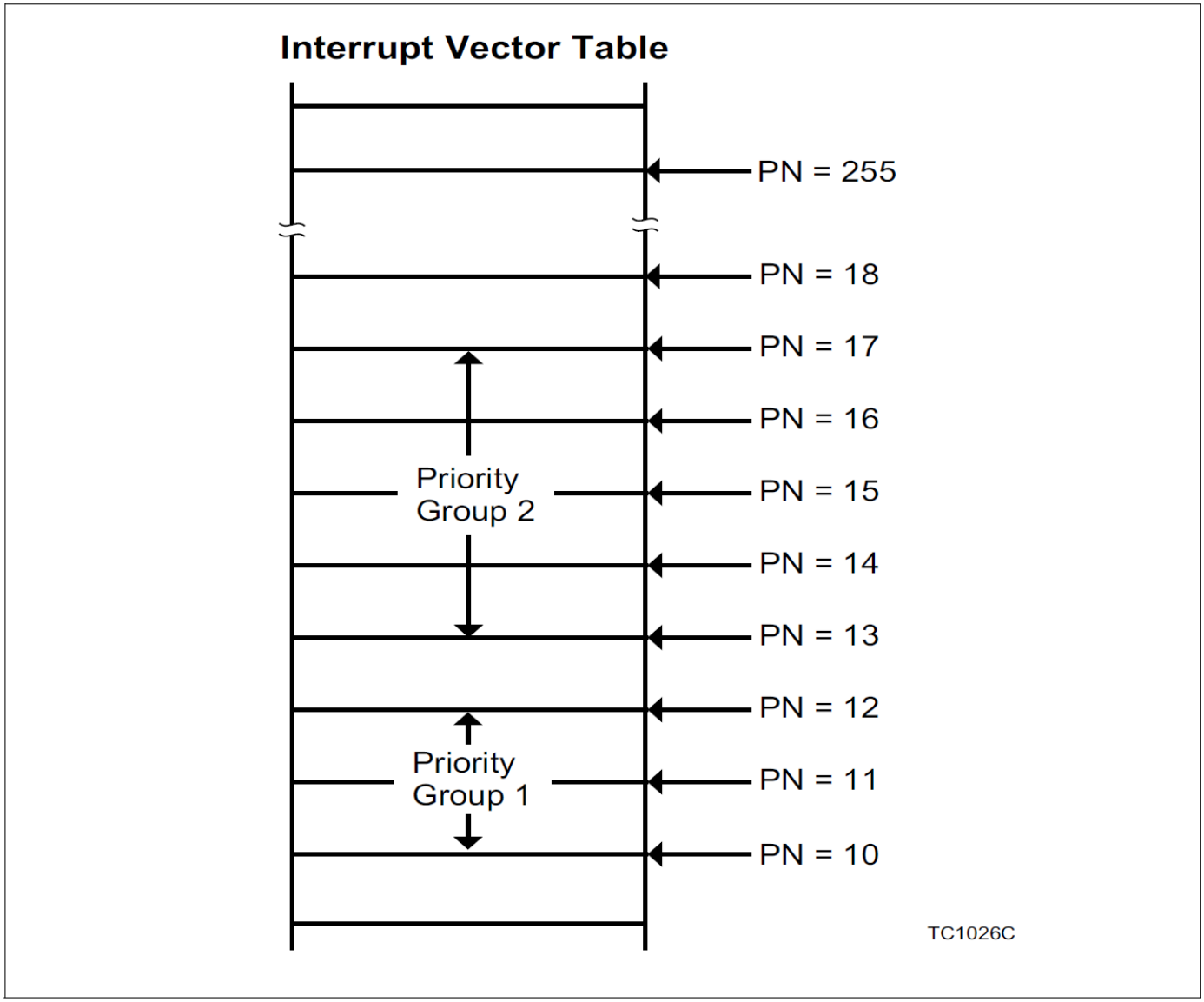


Field	Bits	Type	Description
BIV	[31:1]	rw	**Base Address of Interrupt Vector Table** The address in the BIV register must be aligned to an even byte address (halfword address). Because of the simple ORing of the left-shifted priority number and the contents of the BIV register, the alignment of the base address of the vector table must be to a power of two boundary, dependent on the number of interrupt entries used.
VSS	0	rw	**Vector Spacing Select** 0 : 32 Byte Vector Spacing 1 : 8 Byte Vector Spacing


Interrupt Vector Table
Priority Number
8 Words
PN = 255
8 Words
PN = 5
Service Routine may span several entries
PN = 4
(may not be used if spanned by ISR with PN = 2)
PN = 3
PN = 2
8 Words
PN = 1
8 Words
BIV
PN = 0 (never used)
TC1025D

VSS=0

(5) Interrupt System中断系统



(6) Trap System 陷阱系统

- Trap System（陷阱）
  - trap occurs as a result of an event such as a Non-Maskable Interrupt (NMI), an instruction exception or illegal access.
  - The TriCore architecture contains eight trap classes and these traps are further classified as synchronous or asynchronous, hardware or software.
  - Each trap is assigned a Trap Identification Number (TIN) that identifies the cause of the trap within its class.

(6) Trap System 陷阱系统

- Trap System（陷阱）
  - The entry code for the trap handler is comprised of a vector of code blocks.
  - Each code block provides an entry for one trap.
  - When a trap is taken, the TIN is placed in data register D[15].
  - Eight classes of traps
    - MMU (Memory Management Unit)
    - Internal Protection
    - Instruction Error
    - Context Management
    - System Bus and Peripherals
    - Assertion Trap
    - System Call
    - Non-Maskable Interrupt (NMI)

(6) Trap System 陷阱系统

Table 6-1 Supported Traps

TIN	Name	Synch. / Asynch.	HW / SW	Definition
Class 0 — MMU				
0	VAF	Synch.	HW	Virtual Address Fill.
1	VAP	Synch.	HW	Virtual Address Protection.
Class 1 — Internal Protection Traps				
1	PRIV	Synch.	HW	Privileged Instruction.
2	MPR	Synch.	HW	Memory Protection Read.
3	MPW	Synch.	HW	Memory Protection Write.
4	MPX	Synch.	HW	Memory Protection Execution.
5	MPP	Synch.	HW	Memory Protection Peripheral Access.
6	MPN	Synch.	HW	Memory Protection Null Address.
7	GRWP	Synch.	HW	Global Register Write Protection.
Class 2 — Instruction Errors				
1	IOPC	Synch.	HW	Illegal Opcode.

(6) Trap System 陷阱系统

Table 6-1 Supported Traps (cont'd)

TIN	Name	Synch. / Asynch.	HW / SW	Definition
2	UOPC	Synch.	HW	Unimplemented Opcode.
3	OPD	Synch.	HW	Invalid Operand specification.
4	ALN	Synch.	HW	Data Address Alignment.
5	MEM	Synch.	HW	Invalid Local Memory Address.
Class 3 — Context Management				
1	FCD	Synch.	HW	Free Context List Depletion (FCX = LCX).
2	CDO	Synch.	HW	Call Depth Overflow.
3	CDU	Synch.	HW	Call Depth Underflow.
4	FCU	Synch.	HW	Free Context List Underflow (FCX = 0).
5	CSU	Synch.	HW	Call Stack Underflow (PCX = 0).
6	CTYP	Synch.	HW	Context Type (PCXI.UL wrong).
7	NEST	Synch.	HW	Nesting Error: RFE with non-zero call depth.

(6) Trap System 陷阱系统

Class 4 — System Bus and Peripheral Errors				
1	PSE	Synch.	HW	Program Fetch Synchronous Error.
2	DSE	Synch.	HW	Data Access Synchronous Error.
3	DAE	Asynch.	HW	Data Access Asynchronous Error.
4	CAE	Asynch	HW	Coprocessor Trap Asynchronous Error.TriCore 1.6
5	PIE	Synch	HW	Program Memory Integrity Error.
6	DIE	Asynch	HW	Data Memory Integrity Error. TriCore 1.6
7	TAE	Asynch	HW	Temporal Asynchronous Error
Class 5— Assertion Traps				
1	OVF	Synch.	SW	Arithmetic Overflow.
2	SOVF	Synch.	SW	Sticky Arithmetic Overflow.
Class 6 — System Call[1)]				
	SYS	Synch.	SW	System Call.
Class 7 — Non-Maskable Interrupt				
0	NMI	Asynch.	HW	Non-Maskable Interrupt.

(6) Trap System 陷阱系统

Class 4 — System Bus and Peripheral Errors				
1	PSE	Synch.	HW	Program Fetch Synchronous Error.
2	DSE	Synch.	HW	Data Access Synchronous Error.
3	DAE	Asynch.	HW	Data Access Asynchronous Error.
4	CAE	Asynch	HW	Coprocessor Trap Asynchronous Error.TriCore 1.6
5	PIE	Synch	HW	Program Memory Integrity Error.
6	DIE	Asynch	HW	Data Memory Integrity Error. TriCore 1.6
7	TAE	Asynch	HW	Temporal Asynchronous Error
Class 5— Assertion Traps				
1	OVF	Synch.	SW	Arithmetic Overflow.
2	SOVF	Synch.	SW	Sticky Arithmetic Overflow.
Class 6 — System Call[1)]				
	SYS	Synch.	SW	System Call.
Class 7 — Non-Maskable Interrupt				
0	NMI	Asynch.	HW	Non-Maskable Interrupt.

(6) Trap System 陷阱系统

- Protection System
  - ◆The Trap System.
  - ◆The I/O Privilege System
    - ■ User 0 Mode
    - ■ User 1 Mode : directly access non-critical system peripherals.
    - ■ Supervisor Mode
  - ◆The Memory Protection System

> ➢ Trap System陷阱系统是单片机的内部保护机制之一
> ➢ 除此之外，还有I/O端口优先机机制、内存保护机制等

(7) CPU的通用和系统寄存器

- General Purpose and System Registers
  - ◆CPU的通用和系统寄存器
    - General Purpose Registers
    - System registers (PSW, PC, PCXI)
    - Stack Management registers are (A[10] and ISP)
    - SYSCON and CPU_ID registers
    - Trap registers
    - Context Management registers
    - Memory Protection registers
    - Memory Management registers
    - Debug registers
    - Floating Point registers
    - Special Function registers associated with the core

(7) CPU的通用和系统寄存器

- General Purpose and System Registers

initialization state ⇨ operational state

- Core Special Function Registers (CSFRs) that are only writable in the initialization state are described as ENDINIT protected.
- When in the initialization state, all Core Special Function Registers can be modified, using the MTCR (Move To Core Register) instruction

CSFR寄存器仅在初始化阶段才会被改写

Stack Management Registers

Address register A[10], the Interrupt Stack Pointer (ISP) and a PSW bit are used in the management of the stack.

PSW.IS private stack (PSW.IS == 0),调用A[10] (SP)
private stack (PSW.IS == 1),不调用A[10] (SP)，直接由ISR应用中断堆栈（Interrupt Stack)

A[10](SP)
Address Register A[10] (Stack Pointer)($FFA8_H$)

Reset Value: Implementation Specific

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
A[10](SP)															
rw															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
A[10](SP)															
rw															

Field	Bits	Type	Description
A[10](SP)	[31:0]	rw	Address Register A[10] (Stack Pointer)

Stack Management Registers

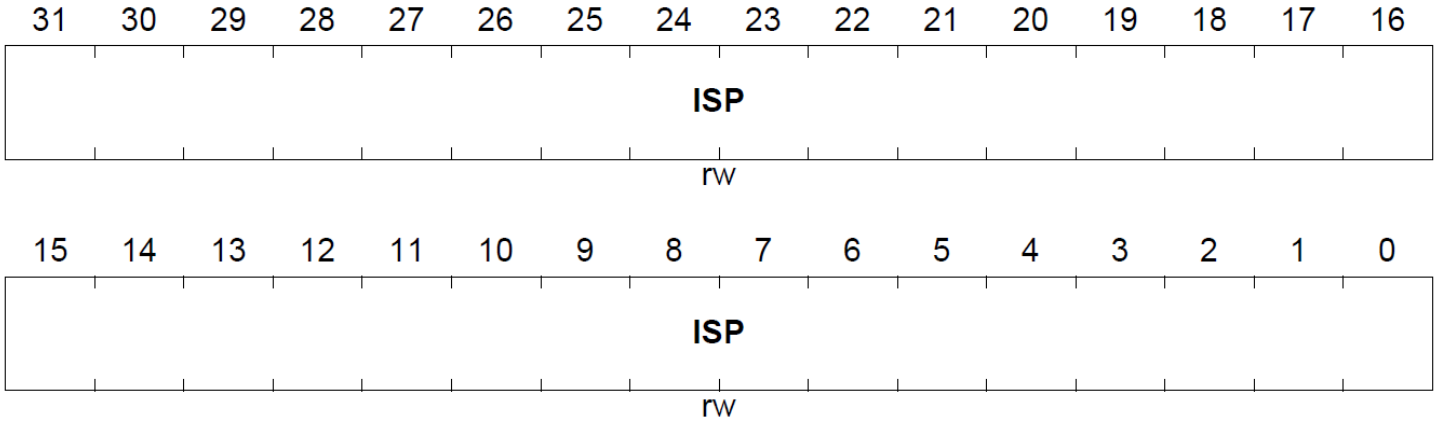
Interrupt Stack Pointer Register (ISP)

The Interrupt Stack Pointer is defined as follows.

Note: This register is ENDINIT protected.

ISP
Interrupt Stack Pointer **($FE28_H$)**

Reset Value: Implementation Specific



Field	Bits	Type	Description
ISP	[31:0]	rw	Interrupt Stack Pointer

(7) CPU的通用和系统寄存器

- System Control Register (SYSCON)

SYSCON
System Configuration Register (FE14$_H$) Reset Value: 0000 0000$_H$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RES														U1_IOS	U1_IED
-														rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RES											TS	IS	TPROTEN	PROTEN	FCDSF
-											rw	rw	rw	rw	rwh

(7) CPU的通用和系统寄存器

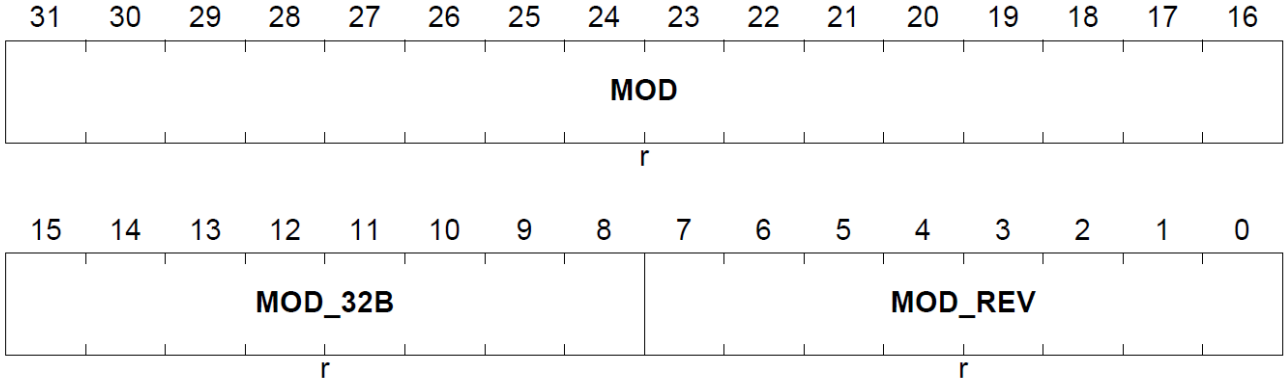
- System Control Register (SYSCON)

Field	Bits	Type	Description
RES	[31:18]	-	**Reserved**
U1_IOS	17	rw	**User-1 Peripheral access as supervisor.** Allow User-1 mode tasks to access peripherals as if in Supervisor mode. Enables User-1 access to all peripheral registers.
U1_IED	16	rw	**User-1 Instruction execution disable.** Disable the execution of User-1 mode instructions in User-1 IO mode. Disables User-1 ability to enable and disable interrupts
RES	[15:5]	-	**Reserved**
TS	4	rw	Initial state of PSW.S bit in trap handler
IS	3	rw	Initial state of PSW.S bit in interrupt handler
TPROTEN	2	rw	**Temporal Protection Enable** Enable the Temporal Protection system. 0 : Temporal Protection is disabled. 1 : Temporal Protection is enabled.
PROTEN	1	rw	**Memory Protection Enable** Enables the memory protection system. Memory protection is controlled through the memory protection register sets. Note: Initialize the protection register sets prior to setting PROTEN to one.

(7) CPU的通用和系统寄存器

● CPU Identification Register (CPU_ID)

CPU_ID
CPU Module Identification **($FE18_H$)**
Reset Value: Implementation Specific



Field	Bits	Type	Description
MOD	[31:16]	r	**Module Identification Number** Used for module identification.
MOD_32B	[15:8]	r	**32-Bit Module Enable** A value of $C0_H$ in this field indicates a 32-bit module with a 32-bit module ID register.
MOD_REV	[7:0]	r	**Module Revision Number** Used for revision numbering. The value of the revision starts at 01_H (first revision) up to FF_H.

(7) CPU的通用和系统寄存器

- Accessing Core Special Function Registers (CSFRs)

  读CSFR寄存器的指令：MFCR (Move From Core Register)

  写CSFR寄存器的指令：MTCR (Move To Core register)

小结

(1) TC212L基本结构

单核、多核、锁步核Lockstep core、CPU、存储空间、总线

(2) 数据类型和(存储空间)寻址方式

- 十进制/十六进制/二进制、Word、变量定义（如单精度浮点数Single-Precision Floating Point），Big/litter-endian byte order
- 地址空间，绝对寻址、相对寻址（Base+offset)

(3) CPU系统寄存器

35个架构寄存器Arhitectural Register：通用目的寄存器GPR、PC/PCXI/PSW

数据寄存器、地址寄存器、全局寄存器、高寄存器、低寄存器

(4) 任务与现场

- 线程tread、任务task、SMT、ISR、Supervisor Mode
- 现场context、现场保存区CSA、链接字Link Word、高/低半现场、A[11]、PCXI
- RET、RFE、FCALL、FRET
- Save CSA、Restore CSA、FCX、PCX

小结

(5) Interrupt System中断系统

- 中断Interrupt、服务请求Service Request(SR)、陷阱Traps、函数调用Function call
- Service Request Node, Direct Memory Access (DMA)
- SRPN, ICU(Interrupt Control Unit), ICR(Interrupt Control Register), CCPN, PIPN
- ISR(Interrupt service routine)
- Interrupt vector, BIV, VSS

(6) Trap System 陷阱系统

- TIN(Trap Identification Number), NMI(non-maskable interrupt), instruction exception, illegal access
- protection system, trap system, I/O privilege system, memory protection system

(7) CPU的通用和系统寄存器

- CSFR(core special function register), ENDINIT, MTCR, MFCR
- A[10], ISP, PSW.IS, SYSCON, CPU_ID

本节结束，谢谢！

Thank you！